BADA - TERMUX / ARM64 - SINGLE-FILE

Bada Engine Sources

Unknown-Prior & Chaos Ensemble Forecast

```
psi_i = sqrt(a_i) * exp(i*theta_i)   =>   |psi_i|^2 = a_i
```

zero-preservation . phase-only reweighting . append-only ledger

Lorenz-63 RK4 ensemble . Lyapunov horizon . Mandelbrot escape-time

Honest scope: no Millennium-problem proof; no prediction past the chaos horizon.
beta(p,q)/(x log x) = (x log x)^n is a false identity and is not used.

compiled listing - 2026-06-29

files: bada.c . predictor.bada . engine.html . chaos_forecast.bada . chaos_engine.html

Contents

1.	`bada.c`	688 lines
2.	`predictor.bada`	158 lines
3.	`engine.html`	405 lines
4.	`chaos_forecast.bada`	169 lines
5.	`chaos_engine.html`	355 lines

All five files compile and run as a set. The two HTML consoles are faithful JavaScript ports of the verified Bada numerics and need no server. Numbers in the listings were checked against the C reference: zeros preserved, |psi|^2 = a at machine epsilon (~5.55e-17), entropy 2.07944 to 1.96477 to 1.72057, Lorenz horizon t ~ 21, Lyapunov ~ 0.5.

1. bada.c

The Unknown-Prior Engine reference interpreter (stage-0). Lexer over the closed keyword set, recursive-descent parser, tree-walking interpreter, the entropy-phase cognitive core, the @reviser learning loop, and the engine builtins. Build: gcc -O2 -std=c11 -o bada src/bada.c -lm

```c
/* ============================================================================
 * bada.c -- The Unknown-Prior Engine reference implementation (stage-0)
 *
 * A faithful single-file C reference for the Bada language. Implements:
 *   - lexer over the closed 26-keyword set + operator algebra
 *   - recursive-descent parser -> AST
 *   - tree-walking interpreter over an append-only ledger (TupleSpace)
 *   - the entropy-phase cognitive core:
 *         psi_i = sqrt(a_i) * exp(i*theta_i),  |psi_i|^2 = a_i exactly
 *     => zero-preservation + norm invariance (the two provable properties)
 *   - the @reviser reflective learning loop / grammar-rule commits
 *   - engine builtins (softmax, entropy, unknown_prior, update, cognitive_system,
 *     zeros_of, maxdiff, last_a, ledger, dist, f5, sci, log, exp, sqrt, abs, len,
 *     char_at, length)
 *
 * CLI: bada run|tokens|ast file.bada
 *
 * Honesty note: this implements ONLY the legitimate, verifiable shapes -- max
 * entropy priors, -p log p, and unit-modulus phase. It makes no claim about the
 * Riemann Hypothesis or zeta/Gamma identities. The one mathematical fact relied
 * on (a unit-modulus factor preserves zeros) is checked directly at runtime.
 * ============================================================================*/
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <complex.h>
#include <ctype.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

static void *xmalloc(size_t n){ void*p=malloc(n); if(!p){fprintf(stderr,"OOM\n");exit(1);} return p; }
static void *xrealloc(void*q,size_t n){ void*p=realloc(q,n); if(!p){fprintf(stderr,"OOM\n");exit(1);} return p; }
static char *xstrdup(const char*s){ size_t n=strlen(s)+1; char*p=xmalloc(n); memcpy(p,s,n); return p; }

/* ---------------------------------------------------------------- tokens -- */
typedef enum {
  T_INT,T_FLOAT,T_STRING,T_IDENT,
  T_OMEGA,T_DATABASE,T_TUPLESPACE,T_TUPLESPACE_C,T_ASPERAL,T_STRUCT,
  T_DEF,T_TYPEDEF,T_RETURN,T_IF,T_ELSE,T_WHILE,T_FOR,T_IN,
  T_STATIC,T_DYNAMIC,T_REFINE,T_STAR,T_TRUE,T_FALSE,T_NIL,T_LET,T_PRINT,T_EACH,
  T_REVISER,
  T_BIND,T_ASSIGN,T_COMMIT,T_QUERY,T_ARROW,T_DEFARROW,T_APPEND,T_TILDE,T_SCOPE,T_PIPE,
  T_PLUS,T_MINUS,T_STARC,T_SLASH,T_PCT,
  T_LT,T_LE,T_GT,T_GE,T_EQ,T_NE,T_AND,T_OR,T_NOT,
  T_LPAREN,T_RPAREN,T_LBRACE,T_RBRACE,T_LBRACK,T_RBRACK,
  T_COMMA,T_DOT,T_COLON,T_SEMI,T_AT,T_EOF
} TokKind;

typedef struct { TokKind kind; char*lex; double num; int line; } Token;
typedef struct { const char*src; size_t pos,len; int line; Token*toks; size_t ntok,captok; } Lexer;

static void push_tok(Lexer*L,TokKind k,const char*lex,double num){
  if(L->ntok==L->captok){ L->captok=L->captok?L->captok*2:64; L->toks=xrealloc(L->toks,L->captok*sizeof(Token)); }
  Token t; t.kind=k; t.lex=lex?xstrdup(lex):NULL; t.num=num; t.line=L->line; L->toks[L->ntok++]=t;
}
typedef struct { const char*w; TokKind k; } KW;
static KW KEYWORDS[]={
  {"Omega",T_OMEGA},{"DATABASE",T_DATABASE},{"tuplespace",T_TUPLESPACE},
  {"TupleSpace",T_TUPLESPACE_C},{"asperal",T_ASPERAL},{"struct",T_STRUCT},
  {"def",T_DEF},{"typedef",T_TYPEDEF},{"return",T_RETURN},{"if",T_IF},
  {"else",T_ELSE},{"while",T_WHILE},{"for",T_FOR},{"in",T_IN},
  {"static",T_STATIC},{"dynamic",T_DYNAMIC},{"refine",T_REFINE},{"star",T_STAR},
  {"true",T_TRUE},{"false",T_FALSE},{"nil",T_NIL},{"let",T_LET},
  {"print",T_PRINT},{"each",T_EACH},{NULL,0}
};
static TokKind kw_lookup(const char*s){ for(int i=0;KEYWORDS[i].w;i++) if(!strcmp(KEYWORDS[i].w,s)) return KEYWORDS[i].k; return T_IDENT; }

static int lpk(Lexer*L){ return L->pos<L->len?L->src[L->pos]:-1; }
```

```c
static int lpk2(Lexer*L){ return L->pos+1<L->len?L->src[L->pos+1]:-1; }
static int lpk3(Lexer*L){ return L->pos+2<L->len?L->src[L->pos+2]:-1; }
static int ladv(Lexer*L){ int c=lpk(L); L->pos++; if(c=='\n')L->line++; return c; }

static void lex_all(Lexer*L){
  while(L->pos<L->len){
    int c=lpk(L);
    if(c=='\n'||isspace(c)){ ladv(L); continue; }
    if(c=='#'){ while(lpk(L)!=-1&&lpk(L)!='\n') ladv(L); continue; }
    if(c=='/'&&lpk2(L)=='/'){ while(lpk(L)!=-1&&lpk(L)!='\n') ladv(L); continue; }
    if(c=='@'){
      size_t save=L->pos; ladv(L); char buf[32]; int n=0;
      while(isalpha(lpk(L))&&n<31) buf[n++]=ladv(L); buf[n]=0;
      if(!strcmp(buf,"reviser")) push_tok(L,T_REVISER,"@reviser",0);
      else { L->pos=save; ladv(L); push_tok(L,T_AT,"@",0); }
      continue;
    }
    if(isdigit(c)||(c=='.'&&isdigit(lpk2(L)))){
      size_t s=L->pos; int isf=0;
      while(isdigit(lpk(L))) ladv(L);
      if(lpk(L)=='.'&&isdigit(lpk2(L))){ isf=1; ladv(L); while(isdigit(lpk(L))) ladv(L); }
      if(lpk(L)=='e'||lpk(L)=='E'){ isf=1; ladv(L); if(lpk(L)=='+'||lpk(L)=='-') ladv(L); while(isdigit(lpk(L))) ladv(L); }
      size_t len=L->pos-s; char*t=xmalloc(len+1); memcpy(t,L->src+s,len); t[len]=0;
      push_tok(L,isf?T_FLOAT:T_INT,t,atof(t)); free(t); continue;
    }
    if(isalpha(c)||c=='_'){
      size_t s=L->pos; while(isalnum(lpk(L))||lpk(L)=='_') ladv(L);
      size_t len=L->pos-s; char*t=xmalloc(len+1); memcpy(t,L->src+s,len); t[len]=0;
      push_tok(L,kw_lookup(t),t,0); free(t); continue;
    }
    if(c=='"'){
      ladv(L); size_t cap=16,n=0; char*b=xmalloc(cap);
      while(lpk(L)!=-1&&lpk(L)!='"'){
        int ch=ladv(L);
        if(ch=='\\'){ int e=ladv(L); switch(e){case 'n':ch='\n';break;case 't':ch='\t';break;case '\\':ch='\\';break;case '"':ch='"';break;default:ch=e;} }
        if(n+1>=cap){cap*=2;b=xrealloc(b,cap);} b[n++]=ch;
      }
      ladv(L); b[n]=0; push_tok(L,T_STRING,b,0); free(b); continue;
    }
    #define M3(a,b,cc,K) if(c==a&&lpk2(L)==b&&lpk3(L)==cc){ladv(L);ladv(L);ladv(L);push_tok(L,K,NULL,0);continue;}
    #define M2(a,b,K) if(c==a&&lpk2(L)==b){ladv(L);ladv(L);push_tok(L,K,NULL,0);continue;}
    #define M1(a,K) if(c==a){ladv(L);push_tok(L,K,NULL,0);continue;}
    M3(':','=','>',T_DEFARROW)
    M2(':','=',T_BIND) M2('>','>',T_COMMIT) M2('=','>',T_QUERY) M2('-','>',T_ARROW)
    M2('<','-',T_APPEND) M2(':',':',T_SCOPE) M2('<','=',T_LE) M2('>','=',T_GE)
    M2('=','=',T_EQ) M2('!','=',T_NE) M2('&','&',T_AND) M2('|','|',T_OR)
    M1('=',T_ASSIGN) M1('+',T_PLUS) M1('-',T_MINUS) M1('*',T_STARC) M1('/',T_SLASH) M1('%',T_PCT)
    M1('<',T_LT) M1('>',T_GT) M1('~',T_TILDE) M1('!',T_NOT)
    M1('(',T_LPAREN) M1(')',T_RPAREN) M1('{',T_LBRACE) M1('}',T_RBRACE)
    M1('[',T_LBRACK) M1(']',T_RBRACK) M1(',',T_COMMA) M1('.',T_DOT) M1(':',T_COLON) M1(';',T_SEMI) M1('|',T_PIPE)
    #undef M3
    #undef M2
    #undef M1
    fprintf(stderr,"lex error: unexpected '%c' (line %d)\n",c,L->line); ladv(L);
  }
  push_tok(L,T_EOF,NULL,0);
}

static const char *tok_name(TokKind k){
  switch(k){
    case T_INT:return"INT";case T_FLOAT:return"FLOAT";case T_STRING:return"STRING";case T_IDENT:return"IDENT";
    case T_OMEGA:return"OMEGA";case T_DATABASE:return"DATABASE";case T_TUPLESPACE:return"TUPLESPACE";
    case T_TUPLESPACE_C:return"TUPLESPACE";case T_ASPERAL:return"ASPERAL";case T_STRUCT:return"STRUCT";
    case T_DEF:return"DEF";case T_TYPEDEF:return"TYPEDEF";case T_RETURN:return"RETURN";case T_IF:return"IF";
    case T_ELSE:return"ELSE";case T_WHILE:return"WHILE";case T_FOR:return"FOR";case T_IN:return"IN";
    case T_STATIC:return"STATIC";case T_DYNAMIC:return"DYNAMIC";case T_REFINE:return"REFINE";case T_STAR:return"STAR";
    case T_TRUE:return"TRUE";case T_FALSE:return"FALSE";case T_NIL:return"NIL";case T_LET:return"LET";
    case T_PRINT:return"PRINT";case T_EACH:return"EACH";case T_REVISER:return"REVISER";case T_BIND:return"BIND";
    case T_ASSIGN:return"ASSIGN";case T_COMMIT:return"COMMIT";case T_QUERY:return"QUERY";case T_ARROW:return"ARROW";
    case T_DEFARROW:return"DEFARROW";case T_APPEND:return"APPEND";case T_TILDE:return"TILDE";case T_SCOPE:return"SCOPE";
    case T_PIPE:return"PIPE";case T_PLUS:return"PLUS";case T_MINUS:return"MINUS";case T_STARC:return"STARC";
    case T_SLASH:return"SLASH";case T_PCT:return"PCT";case T_LT:return"LT";case T_LE:return"LE";case T_GT:return"GT";
    case T_GE:return"GE";case T_EQ:return"EQ";case T_NE:return"NE";case T_AND:return"AND";case T_OR:return"OR";
    case T_NOT:return"NOT";case T_LPAREN:return"LPAREN";case T_RPAREN:return"RPAREN";case T_LBRACE:return"LBRACE";
    case T_RBRACE:return"RBRACE";case T_LBRACK:return"LBRACK";case T_RBRACK:return"RBRACK";case T_COMMA:return"COMMA";
    case T_DOT:return"DOT";case T_COLON:return"COLON";case T_SEMI:return"SEMI";case T_AT:return"AT";case T_EOF:return"EOF";
    default:return"?";
```

```c
  }
}

/* ---------------------------------------------------------------------- ast -- */
typedef enum {
  N_INT,N_FLOAT,N_STR,N_BOOL,N_NIL,N_STARLIT,N_IDENT,N_OMEGA,N_TUPLESPACE,
  N_ARRAY,N_LAMBDA,N_BIND,N_ASSIGN,N_CALL,N_INDEX,N_MEMBER,N_SCOPE,N_UNARY,N_BINARY,
  N_COMMIT,N_APPEND,N_QUERY,N_IF,N_WHILE,N_FOR,N_RETURN,N_PRINT,N_BLOCK,
  N_FUNC,N_ASPERAL,N_TYPEDEF,N_OMEGABLOCK,N_EACH,N_REVISER,N_RULE,N_EXTENDED,N_PROGRAM
} NodeKind;
typedef struct Node Node;
struct Node { NodeKind kind; double num; char*str; Node**kids; int nkid; char**params; int nparam; int line; };
static Node*mknode(NodeKind k){ Node*n=xmalloc(sizeof(Node)); memset(n,0,sizeof(Node)); n->kind=k; return n; }
static void addkid(Node*n,Node*c){ n->kids=xrealloc(n->kids,(n->nkid+1)*sizeof(Node*)); n->kids[n->nkid++]=c; }
static void addparam(Node*n,const char*p){ n->params=xrealloc(n->params,(n->nparam+1)*sizeof(char*)); n->params[n->npa
ram++]=xstrdup(p); }

typedef struct { Token*t; size_t i,n; } Parser;
static Token*cur(Parser*P){ return &P->t[P->i]; }
static TokKind curk(Parser*P){ return P->t[P->i].kind; }
static Token*padv(Parser*P){ return &P->t[P->i++]; }
static int accept(Parser*P,TokKind k){ if(curk(P)==k){P->i++;return 1;} return 0; }
static void expect(Parser*P,TokKind k,const char*what){
  if(curk(P)!=k){ fprintf(stderr,"parse error: expected %s but got %s (line %d)\n",what,tok_name(curk(P)),cur(P)->line
); exit(1); }
  P->i++;
}
static Node*parse_expr(Parser*P);
static Node*parse_block(Parser*P);
static Node*parse_decl(Parser*P);
static Node*parse_stmt(Parser*P);

static void parse_pipe_params(Parser*P,Node*fn){
  expect(P,T_PIPE,"|");
  if(curk(P)!=T_PIPE){ do{ if(curk(P)==T_IDENT){ addparam(fn,cur(P)->lex); padv(P);} else break; }while(accept(P,T_COM
MA)); }
  expect(P,T_PIPE,"|");
}
static Node*parse_do_or_expr(Parser*P){
  if(curk(P)==T_IDENT&&!strcmp(cur(P)->lex,"do")){
    padv(P); Node*blk=mknode(N_BLOCK);
    while(!(curk(P)==T_IDENT&&!strcmp(cur(P)->lex,"end"))&&curk(P)!=T_EOF) addkid(blk,parse_stmt(P));
    if(curk(P)==T_IDENT&&!strcmp(cur(P)->lex,"end")) padv(P);
    return blk;
  }
  return parse_expr(P);
}
static Node*parse_array(Parser*P){
  expect(P,T_LBRACK,"["); Node*a=mknode(N_ARRAY);
  if(curk(P)!=T_RBRACK){ do{ addkid(a,parse_expr(P)); }while(accept(P,T_COMMA)); }
  expect(P,T_RBRACK,"]"); return a;
}
static Node*parse_lambda(Parser*P){ Node*fn=mknode(N_LAMBDA); parse_pipe_params(P,fn); addkid(fn,parse_do_or_expr(P));
 return fn; }

static Node*parse_primary(Parser*P){
  Token*t=cur(P);
  switch(t->kind){
    case T_INT:{ Node*n=mknode(N_INT); n->num=t->num; padv(P); return n; }
    case T_FLOAT:{ Node*n=mknode(N_FLOAT); n->num=t->num; padv(P); return n; }
    case T_STRING:{ Node*n=mknode(N_STR); n->str=xstrdup(t->lex); padv(P); return n; }
    case T_TRUE:{ Node*n=mknode(N_BOOL); n->num=1; padv(P); return n; }
    case T_FALSE:{ Node*n=mknode(N_BOOL); n->num=0; padv(P); return n; }
    case T_NIL:{ padv(P); return mknode(N_NIL); }
    case T_STAR:{ padv(P); return mknode(N_STARLIT); }
    case T_OMEGA:{ padv(P); return mknode(N_OMEGA); }
    case T_TUPLESPACE: case T_TUPLESPACE_C:{ padv(P); return mknode(N_TUPLESPACE); }
    case T_IDENT:{ Node*n=mknode(N_IDENT); n->str=xstrdup(t->lex); padv(P); return n; }
    case T_LBRACK: return parse_array(P);
    case T_PIPE: return parse_lambda(P);
    case T_LPAREN:{ padv(P); Node*e=parse_expr(P); expect(P,T_RPAREN,")"); return e; }
    default:
      fprintf(stderr,"parse error: unexpected %s in expression (line %d)\n",tok_name(t->kind),t->line); exit(1);
  }
}
static Node*parse_postfix(Parser*P){
  Node*e=parse_primary(P);
  for(;;){
    if(curk(P)==T_LPAREN){
      padv(P); Node*c=mknode(N_CALL); addkid(c,e);
      if(curk(P)!=T_RPAREN){ do{ addkid(c,parse_expr(P)); }while(accept(P,T_COMMA)); }
```

```
      expect(P,T_RPAREN,")"); e=c;
    } else if(curk(P)==T_LBRACK){
      padv(P); Node*ix=mknode(N_INDEX); addkid(ix,e); addkid(ix,parse_expr(P)); expect(P,T_RBRACK,"]"); e=ix;
    } else if(curk(P)==T_DOT){
      padv(P);
      if(curk(P)==T_EACH){
        padv(P); Node*ea=mknode(N_EACH); addkid(ea,e);
        Node*fn=mknode(N_LAMBDA); if(curk(P)==T_PIPE) parse_pipe_params(P,fn); addkid(fn,parse_do_or_expr(P));
        addkid(ea,fn); e=ea;
      } else {
        Node*m=mknode(N_MEMBER); addkid(m,e); m->str=xstrdup(cur(P)->lex); expect(P,T_IDENT,"member name");
        if(curk(P)==T_LPAREN){
          padv(P); Node*c=mknode(N_CALL); addkid(c,m);
          if(curk(P)!=T_RPAREN){ do{ addkid(c,parse_expr(P)); }while(accept(P,T_COMMA)); }
          expect(P,T_RPAREN,")"); e=c;
        } else e=m;
      }
    } else if(curk(P)==T_SCOPE){
      padv(P); Node*s=mknode(N_SCOPE); addkid(s,e);
      if(curk(P)==T_DATABASE){ s->str=xstrdup("DATABASE"); padv(P); }
      else if(curk(P)==T_TUPLESPACE||curk(P)==T_TUPLESPACE_C){ s->str=xstrdup("tuplespace"); padv(P); }
      else { s->str=xstrdup(cur(P)->lex); expect(P,T_IDENT,"scope name"); }
      if(curk(P)==T_LBRACK){ padv(P); Node*r=mknode(N_STR); r->str=xstrdup(cur(P)->lex?cur(P)->lex:"region"); padv(P); expect(P,T_RBRACK,"]"); addkid(s,r); }
      e=s;
    } else break;
  }
  return e;
}
static Node*parse_unary(Parser*P){
  if(curk(P)==T_MINUS||curk(P)==T_NOT){ Node*u=mknode(N_UNARY); u->str=xstrdup(curk(P)==T_MINUS?"-":"!"); padv(P); addkid(u,parse_unary(P)); return u; }
  return parse_postfix(P);
}
static Node*bin(const char*op,Node*a,Node*b){ Node*n=mknode(N_BINARY); n->str=xstrdup(op); addkid(n,a); addkid(n,b); return n; }
static Node*parse_mul(Parser*P){ Node*e=parse_unary(P); while(curk(P)==T_STARC||curk(P)==T_SLASH||curk(P)==T_PCT){ const char*o=curk(P)==T_STARC?"*":curk(P)==T_SLASH?"/":"%"; padv(P); e=bin(o,e,parse_unary(P)); } return e; }
static Node*parse_add(Parser*P){ Node*e=parse_mul(P); while(curk(P)==T_PLUS||curk(P)==T_MINUS){ const char*o=curk(P)==T_PLUS?"+":"-"; padv(P); e=bin(o,e,parse_mul(P)); } return e; }
static Node*parse_cmp(Parser*P){ Node*e=parse_add(P); while(curk(P)==T_LT||curk(P)==T_LE||curk(P)==T_GT||curk(P)==T_GE){ const char*o=curk(P)==T_LT?"<":curk(P)==T_LE?"<=":curk(P)==T_GT?">":">="; padv(P); e=bin(o,e,parse_add(P)); } return e; }
static Node*parse_eq(Parser*P){ Node*e=parse_cmp(P); while(curk(P)==T_EQ||curk(P)==T_NE||curk(P)==T_TILDE){ const char*o=curk(P)==T_EQ?"==":curk(P)==T_NE?"!=":"~"; padv(P); e=bin(o,e,parse_cmp(P)); } return e; }
static Node*parse_and(Parser*P){ Node*e=parse_eq(P); while(curk(P)==T_AND){ padv(P); e=bin("&&",e,parse_eq(P)); } return e; }
static Node*parse_or(Parser*P){ Node*e=parse_and(P); while(curk(P)==T_OR){ padv(P); e=bin("||",e,parse_and(P)); } return e; }
static Node*parse_query(Parser*P){ Node*e=parse_or(P); while(curk(P)==T_QUERY||curk(P)==T_ARROW){ padv(P); Node*q=mknode(N_QUERY); addkid(q,e); addkid(q,parse_or(P)); e=q; } return e; }
static Node*parse_commit(Parser*P){ Node*e=parse_query(P); while(curk(P)==T_COMMIT||curk(P)==T_APPEND){ int isC=(curk(P)==T_COMMIT); padv(P); Node*c=mknode(isC?N_COMMIT:N_APPEND); addkid(c,e); addkid(c,parse_query(P)); e=c; } return e; }
static Node*parse_expr(Parser*P){ return parse_commit(P); }

static Node*parse_block(Parser*P){ expect(P,T_LBRACE,"{"); Node*b=mknode(N_BLOCK); while(curk(P)!=T_RBRACE&&curk(P)!=T_EOF) addkid(b,parse_decl(P)); expect(P,T_RBRACE,"}"); return b; }
static Node*parse_if(Parser*P){
  expect(P,T_IF,"if"); int par=accept(P,T_LPAREN); Node*n=mknode(N_IF); addkid(n,parse_expr(P));
  if(par) expect(P,T_RPAREN,")"); addkid(n,parse_block(P));
  if(accept(P,T_ELSE)){ if(curk(P)==T_IF) addkid(n,parse_if(P)); else addkid(n,parse_block(P)); }
  return n;
}
static Node*parse_while(Parser*P){ expect(P,T_WHILE,"while"); int par=accept(P,T_LPAREN); Node*n=mknode(N_WHILE); addkid(n,parse_expr(P)); if(par) expect(P,T_RPAREN,")"); addkid(n,parse_block(P)); return n; }
static Node*parse_for(Parser*P){ expect(P,T_FOR,"for"); Node*n=mknode(N_FOR); n->str=xstrdup(cur(P)->lex); expect(P,T_IDENT,"loop var"); expect(P,T_IN,"in"); addkid(n,parse_expr(P)); addkid(n,parse_block(P)); return n; }
static Node*parse_print(Parser*P){
  expect(P,T_PRINT,"print"); int par=accept(P,T_LPAREN); Node*n=mknode(N_PRINT);
  if(curk(P)!=T_RPAREN&&curk(P)!=T_SEMI&&curk(P)!=T_RBRACE){ do{ addkid(n,parse_expr(P)); }while(accept(P,T_COMMA)); }
  if(par) expect(P,T_RPAREN,")"); return n;
}
static Node*parse_stmt(Parser*P){
  switch(curk(P)){
    case T_IF: return parse_if(P);
    case T_WHILE: return parse_while(P);
    case T_FOR: return parse_for(P);
    case T_PRINT:{ Node*n=parse_print(P); accept(P,T_SEMI); return n; }
    case T_RETURN:{ padv(P); Node*n=mknode(N_RETURN);
      if(curk(P)!=T_RBRACE&&curk(P)!=T_SEMI&&!(curk(P)==T_IDENT&&!strcmp(cur(P)->lex,"end"))) addkid(n,parse_expr(P));
```

```
      accept(P,T_SEMI); return n; }
    default: break;
  }
  if(curk(P)==T_IDENT&&P->t[P->i+1].kind==T_BIND){ Node*n=mknode(N_BIND); n->str=xstrdup(cur(P)->lex); padv(P); padv(P); addkid(n,parse_expr(P)); accept(P,T_SEMI); return n; }
  { Node*lhs=parse_expr(P);
    if(curk(P)==T_ASSIGN){ padv(P); Node*n=mknode(N_ASSIGN); addkid(n,lhs); addkid(n,parse_expr(P)); accept(P,T_SEMI); return n; }
    accept(P,T_SEMI); return lhs; }
}
static Node*parse_func(Parser*P){
  expect(P,T_DEF,"def"); Node*fn=mknode(N_FUNC); fn->str=xstrdup(cur(P)->lex); expect(P,T_IDENT,"function name");
  if(curk(P)==T_PIPE) parse_pipe_params(P,fn);
  else if(curk(P)==T_LPAREN){ padv(P); if(curk(P)!=T_RPAREN){ do{ addparam(fn,cur(P)->lex); expect(P,T_IDENT,"param"); }while(accept(P,T_COMMA)); } expect(P,T_RPAREN,")"); }
  if(curk(P)==T_LBRACE){ addkid(fn,parse_block(P)); }
  else { if(curk(P)==T_COMMIT||curk(P)==T_QUERY||curk(P)==T_DEFARROW||curk(P)==T_ASSIGN) padv(P);
    Node*ret=mknode(N_RETURN); addkid(ret,parse_expr(P)); Node*blk=mknode(N_BLOCK); addkid(blk,ret); addkid(fn,blk); accept(P,T_SEMI); }
  return fn;
}
static Node*parse_asperal(Parser*P){
  expect(P,T_ASPERAL,"asperal"); Node*a=mknode(N_ASPERAL);
  if(curk(P)==T_PIPE){ parse_pipe_params(P,a); accept(P,T_DEFARROW); addkid(a,parse_block(P)); }
  else { a->str=xstrdup(cur(P)->lex?cur(P)->lex:"anon");
    if(curk(P)==T_IDENT||curk(P)==T_TUPLESPACE_C||curk(P)==T_TUPLESPACE) padv(P);
    accept(P,T_COLON); accept(P,T_STRUCT);
    while(curk(P)!=T_LBRACE&&curk(P)!=T_EOF) padv(P); addkid(a,parse_block(P)); }
  return a;
}
static Node*parse_typedef(Parser*P){ expect(P,T_TYPEDEF,"typedef"); Node*n=mknode(N_TYPEDEF); while(curk(P)!=T_LBRACE&&curk(P)!=T_EOF) padv(P); addkid(n,parse_block(P)); return n; }
static Node*parse_omega_block(Parser*P){
  expect(P,T_OMEGA,"Omega"); expect(P,T_SCOPE,"::"); Node*n=mknode(N_OMEGABLOCK);
  if(curk(P)==T_DATABASE){ n->str=xstrdup("DATABASE"); padv(P); }
  else if(curk(P)==T_TUPLESPACE||curk(P)==T_TUPLESPACE_C){ n->str=xstrdup("tuplespace"); padv(P); }
  else { n->str=xstrdup("DATABASE"); }
  if(curk(P)==T_LBRACK){ padv(P); if(curk(P)!=T_RBRACK) padv(P); expect(P,T_RBRACK,"]"); }
  if(curk(P)==T_LT){ padv(P); if(curk(P)==T_IDENT||curk(P)==T_DATABASE) padv(P); }
  addkid(n,parse_block(P)); return n;
}
static Node*parse_rule(Parser*P){
  padv(P); Node*r=mknode(N_RULE); r->str=xstrdup(cur(P)->lex?cur(P)->lex:"r"); padv(P);
  Node*head=mknode(N_STR); head->str=xstrdup(cur(P)->lex?cur(P)->lex:"stmt"); padv(P); addkid(r,head);
  Node*pat=mknode(N_ARRAY); expect(P,T_LBRACK,"[");
  while(curk(P)!=T_RBRACK&&curk(P)!=T_EOF){ Node*tk=mknode(N_STR);
    if(curk(P)==T_STRING) tk->str=xstrdup(cur(P)->lex); else if(cur(P)->lex) tk->str=xstrdup(cur(P)->lex); else tk->str=xstrdup(tok_name(curk(P)));
    padv(P); addkid(pat,tk); }
  expect(P,T_RBRACK,"]"); addkid(r,pat);
  accept(P,T_QUERY); accept(P,T_ARROW); accept(P,T_DEFARROW);
  addkid(r,parse_expr(P)); accept(P,T_SEMI); return r;
}
static Node*parse_reviser(Parser*P){
  expect(P,T_REVISER,"@reviser"); Node*n=mknode(N_REVISER);
  while(curk(P)!=T_LBRACE&&curk(P)!=T_EOF) padv(P); expect(P,T_LBRACE,"{");
  while(curk(P)!=T_RBRACE&&curk(P)!=T_EOF){
    if(curk(P)==T_IDENT&&!strcmp(cur(P)->lex,"rule")) addkid(n,parse_rule(P));
    else addkid(n,parse_decl(P));
  }
  expect(P,T_RBRACE,"}"); return n;
}
static Node*parse_decl(Parser*P){
  switch(curk(P)){
    case T_OMEGA:
      if(P->t[P->i+1].kind==T_SCOPE&&(P->t[P->i+2].kind==T_DATABASE||P->t[P->i+2].kind==T_TUPLESPACE||P->t[P->i+2].kind==T_TUPLESPACE_C)) return parse_omega_block(P);
      break;
    case T_TUPLESPACE_C: case T_TUPLESPACE:
      if(P->t[P->i+1].kind==T_LT){ Node*n=mknode(N_OMEGABLOCK); n->str=xstrdup("tuplespace"); padv(P); padv(P); if(curk(P)==T_DATABASE||curk(P)==T_IDENT) padv(P); addkid(n,parse_block(P)); return n; }
      break;
    case T_ASPERAL: return parse_asperal(P);
    case T_TYPEDEF: return parse_typedef(P);
    case T_DEF: return parse_func(P);
    case T_REVISER: return parse_reviser(P);
    default: break;
  }
  return parse_stmt(P);
}
static Node*parse_program(Parser*P){ Node*prog=mknode(N_PROGRAM); while(curk(P)!=T_EOF) addkid(prog,parse_decl(P)); re
```

```
turn prog; }

/* ---------------------------------------------------- values & environment -- */
typedef enum { V_NIL,V_NUM,V_BOOL,V_STR,V_ARR,V_FUN,V_BUILTIN,V_STAR,V_LEDGER } VKind;
typedef struct Value Value;
typedef struct Env Env;
typedef struct { Value*items; int n,cap; } Arr;
struct Value { VKind kind; double num; int b; char*str; Arr*arr; Node*fn; Env*clo; int builtin; };
static Value VNIL(){ Value v; v.kind=V_NIL; return v; }
static Value VNUM(double x){ Value v; v.kind=V_NUM; v.num=x; return v; }
static Value VBOOL(int b){ Value v; v.kind=V_BOOL; v.b=b; return v; }
static Value VSTR(const char*s){ Value v; v.kind=V_STR; v.str=xstrdup(s); return v; }
static Value VSTAR(){ Value v; v.kind=V_STAR; return v; }
static Arr*arr_new(){ Arr*a=xmalloc(sizeof(Arr)); a->items=NULL; a->n=0; a->cap=0; return a; }
static void arr_push(Arr*a,Value v){ if(a->n==a->cap){ a->cap=a->cap?a->cap*2:8; a->items=xrealloc(a->items,a->cap*sizeof(Value)); } a->items[a->n++]=v; }
static Value VARR(Arr*a){ Value v; v.kind=V_ARR; v.arr=a; return v; }

typedef struct Binding { char*name; Value val; struct Binding*next; } Binding;
struct Env { Binding*head; Env*parent; };
static Env*env_new(Env*p){ Env*e=xmalloc(sizeof(Env)); e->head=NULL; e->parent=p; return e; }
static void env_bind(Env*e,const char*name,Value v){ Binding*b=xmalloc(sizeof(Binding)); b->name=xstrdup(name); b->val=v; b->next=e->head; e->head=b; }
static Binding*env_find(Env*e,const char*name){ for(Env*s=e;s;s=s->parent) for(Binding*b=s->head;b;b=b->next) if(!strcmp(b->name,name)) return b; return NULL; }

typedef struct { char*region; Value v; } Fact;
typedef struct { Fact*facts; int n,cap; } Ledger;
static Ledger LEDGER={0,0,0};
static int ledger_append(const char*region,Value v){
  if(LEDGER.n==LEDGER.cap){ LEDGER.cap=LEDGER.cap?LEDGER.cap*2:16; LEDGER.facts=xrealloc(LEDGER.facts,LEDGER.cap*sizeof(Fact)); }
  LEDGER.facts[LEDGER.n].region=xstrdup(region?region:"tuplespace"); LEDGER.facts[LEDGER.n].v=v; return LEDGER.n++;
}
typedef struct { char*name,*head; Node*pattern; Node*denot; } Rule;
typedef struct { Rule*rules; int n,cap; } RuleLedger;
static RuleLedger GRAMMAR={0,0,0};
static void grammar_commit(const char*name,const char*head,Node*pat,Node*den){
  if(GRAMMAR.n==GRAMMAR.cap){ GRAMMAR.cap=GRAMMAR.cap?GRAMMAR.cap*2:8; GRAMMAR.rules=xrealloc(GRAMMAR.rules,GRAMMAR.cap*sizeof(Rule)); }
  Rule*r=&GRAMMAR.rules[GRAMMAR.n++]; r->name=xstrdup(name); r->head=xstrdup(head); r->pattern=pat; r->denot=den;
}

/* ---------------------------------------------------- engine core math -- */
static void softmax_vec(const double*z,int n,double*out){
  double mx=z[0]; for(int i=1;i<n;i++) if(z[i]>mx) mx=z[i];
  double s=0;
  for(int i=0;i<n;i++){
    if(z[i]==-INFINITY||(mx-z[i])>30.0) out[i]=0.0; else out[i]=exp(z[i]-mx);
    s+=out[i];
  }
  for(int i=0;i<n;i++) out[i]/=s;
}
static double shannon_entropy(const double*q,int n){ double h=0; for(int i=0;i<n;i++){ double x=q[i]; if(x>1e-12) h-=x*log(x); } return h; }
static void cognitive_system(const double*z,int V,double**qm,double**phim,double*beta,int M,double*q_hat,double*maxdiff){
  double*a=xmalloc(V*sizeof(double)); softmax_vec(z,V,a);
  double*theta=xmalloc(V*sizeof(double)); for(int i=0;i<V;i++) theta[i]=0.0;
  for(int m=0;m<M;m++){ double H=shannon_entropy(qm[m],V); for(int i=0;i<V;i++) theta[i]+=beta[m]*H*phim[m][i]; }
  double sum=0; double*mag2=xmalloc(V*sizeof(double));
  for(int i=0;i<V;i++){ double complex psi=sqrt(a[i])*cexp(I*theta[i]); mag2[i]=creal(psi*conj(psi)); sum+=mag2[i]; }
  double md=0; for(int i=0;i<V;i++){ q_hat[i]=mag2[i]/sum; double d=fabs(q_hat[i]-a[i]); if(d>md) md=d; }
  *maxdiff=md; free(a); free(theta); free(mag2);
}

/* ---------------------------------------------------- interpreter -- */
typedef struct { int is_return; Value val; } Eval;
static Value eval_node(Node*n,Env*e);
static Eval exec_block(Node*blk,Env*e);
static int truthy(Value v){ switch(v.kind){ case V_NIL:return 0; case V_BOOL:return v.b; case V_NUM:return v.num!=0; case V_STR:return v.str&&v.str[0]; case V_ARR:return v.arr&&v.arr->n>0; default:return 1; } }
static int val_eq(Value a,Value b){
  if(a.kind==V_STAR||b.kind==V_STAR) return 0;
  if(a.kind!=b.kind){ if(a.kind==V_NUM&&b.kind==V_BOOL) return a.num==b.b; if(a.kind==V_BOOL&&b.kind==V_NUM) return a.b==b.num; return 0; }
  switch(a.kind){ case V_NUM:return a.num==b.num; case V_BOOL:return a.b==b.b; case V_STR:return !strcmp(a.str,b.str); case V_NIL:return 1; default:return 0; }
}
static void print_val(Value v){
  switch(v.kind){
```

```
    case V_NIL: printf("nil"); break;
    case V_BOOL: printf(v.b?"true":"false"); break;
    case V_STAR: printf("star"); break;
    case V_STR: printf("%s",v.str); break;
    case V_NUM:{ double r=v.num; if(r==(long long)r&&fabs(r)<1e15) printf("%lld",(long long)r); else printf("%g",r); } break;
    case V_ARR:{ printf("["); for(int i=0;i<v.arr->n;i++){ if(i)printf(", "); print_val(v.arr->items[i]); } printf("]"); } break;
    case V_FUN: printf("<fn>"); break;
    case V_BUILTIN: printf("<builtin>"); break;
    case V_LEDGER: printf("<ledger>"); break;
  }
}
enum { B_SOFTMAX,B_ENTROPY,B_UNKNOWN_PRIOR,B_UPDATE,B_MANIFOLD_EMBED,B_COGNITIVE,
  B_DIST,B_ZEROS_OF,B_MAXDIFF,B_LAST_A,B_LEDGER,B_LEN,B_SQRT,B_LOG,B_EXP,B_ABS,
  B_F5,B_SCI,B_CHAR_AT,B_STRLEN,B_CORREL,
  B_SIN,B_COS,B_POW,B_FLOOR,B_MAX,B_MIN,B_STDEV,B_MEAN,B_NBUILTIN };
static const char*BNAMES[B_NBUILTIN]={
  "softmax","entropy","unknown_prior","update","manifold_embed","cognitive_system",
  "dist","zeros_of","maxdiff","last_a","ledger","len","sqrt","log","exp","abs",
  "f5","sci","char_at","length","correlation",
  "sin","cos","pow","floor","fmax","fmin","stdev","mean" };
static double G_LAST_MAXDIFF=0.0;
static Arr*G_LAST_A=NULL;
static Value num_arr_to_val(const double*x,int n){ Arr*a=arr_new(); for(int i=0;i<n;i++) arr_push(a,VNUM(x[i])); return VARR(a); }
static double*val_to_num_arr(Value v,int*n){ if(v.kind!=V_ARR){ *n=0; return NULL; } *n=v.arr->n; double*x=xmalloc((*n>0?*n:1)*sizeof(double)); for(int i=0;i<*n;i++) x[i]=v.arr->items[i].kind==V_NUM?v.arr->items[i].num:0.0; return x; }
static Value call_builtin(int idx,Value*args,int nargs);
static Value call_function(Value fn,Value*args,int nargs){
  if(fn.kind==V_BUILTIN) return call_builtin(fn.builtin,args,nargs);
  if(fn.kind!=V_FUN){ fprintf(stderr,"runtime error: not callable\n"); exit(1); }
  Env*local=env_new(fn.clo); Node*node=fn.fn;
  for(int i=0;i<node->nparam;i++) env_bind(local,node->params[i],i<nargs?args[i]:VNIL());
  Node*body=node->kids[node->nkid-1]; Eval r=exec_block(body,local); return r.is_return?r.val:VNIL();
}
static void assign_lvalue(Node*lhs,Value v,Env*e){
  if(lhs->kind==N_IDENT){ Binding*b=env_find(e,lhs->str); if(b) b->val=v; else env_bind(e,lhs->str,v); return; }
  if(lhs->kind==N_INDEX){ Value base=eval_node(lhs->kids[0],e); Value idx=eval_node(lhs->kids[1],e);
    if(base.kind==V_ARR&&idx.kind==V_NUM){ int i=(int)idx.num; if(i>=0&&i<base.arr->n) base.arr->items[i]=v; } return; }
  fprintf(stderr,"runtime error: bad assignment target\n"); exit(1);
}
static Value eval_extended(Node*n,Env*e){
  for(int r=GRAMMAR.n-1;r>=0;r--) if(!strcmp(GRAMMAR.rules[r].name,n->str)){
    Env*local=env_new(e);
    for(int i=0;i<n->nkid;i++){ char hole[16]; snprintf(hole,16,"_%d",i+1); env_bind(local,hole,eval_node(n->kids[i],e)); }
    return eval_node(GRAMMAR.rules[r].denot,local);
  }
  return VNIL();
}
static Value eval_node(Node*n,Env*e){
  switch(n->kind){
    case N_INT: case N_FLOAT: return VNUM(n->num);
    case N_BOOL: return VBOOL((int)n->num);
    case N_STR: return VSTR(n->str);
    case N_NIL: return VNIL();
    case N_STARLIT: return VSTAR();
    case N_OMEGA: case N_TUPLESPACE:{ Value v; v.kind=V_LEDGER; return v; }
    case N_IDENT:{ Binding*b=env_find(e,n->str); if(b) return b->val;
      for(int i=0;i<B_NBUILTIN;i++) if(!strcmp(BNAMES[i],n->str)){ Value v; v.kind=V_BUILTIN; v.builtin=i; return v; }
      return VNIL(); }
    case N_ARRAY:{ Arr*a=arr_new(); for(int i=0;i<n->nkid;i++) arr_push(a,eval_node(n->kids[i],e)); return VARR(a); }
    case N_LAMBDA:{ Value v; v.kind=V_FUN; v.fn=n; v.clo=e; return v; }
    case N_BIND:{ Value v=eval_node(n->kids[0],e); env_bind(e,n->str,v); return v; }
    case N_ASSIGN:{ Value v=eval_node(n->kids[1],e); assign_lvalue(n->kids[0],v,e); return v; }
    case N_UNARY:{ Value a=eval_node(n->kids[0],e); if(!strcmp(n->str,"-")) return VNUM(-(a.kind==V_NUM?a.num:0)); return VBOOL(!truthy(a)); }
    case N_BINARY:{
      const char*o=n->str;
      if(!strcmp(o,"&&")){ Value a=eval_node(n->kids[0],e); if(!truthy(a)) return VBOOL(0); return VBOOL(truthy(eval_node(n->kids[1],e))); }
      if(!strcmp(o,"||")){ Value a=eval_node(n->kids[0],e); if(truthy(a)) return VBOOL(1); return VBOOL(truthy(eval_node(n->kids[1],e))); }
      Value a=eval_node(n->kids[0],e),b=eval_node(n->kids[1],e);
      if(!strcmp(o,"~")){ if(a.kind==V_STAR||b.kind==V_STAR) return VBOOL(1); return VBOOL(val_eq(a,b)); }
      if(!strcmp(o,"==")) return VBOOL(val_eq(a,b));
      if(!strcmp(o,"!=")) return VBOOL(!val_eq(a,b));
      double x=a.kind==V_NUM?a.num:0,y=b.kind==V_NUM?b.num:0;
      if(!strcmp(o,"+")){
```

```
        if(a.kind==V_STR||b.kind==V_STR){ char buf[512],la[128],lb[128];
          if(a.kind==V_STR) snprintf(la,128,"%s",a.str); else snprintf(la,128,"%g",x);
          if(b.kind==V_STR) snprintf(lb,128,"%s",b.str); else snprintf(lb,128,"%g",y);
          snprintf(buf,512,"%s%s",la,lb); return VSTR(buf); }
        return VNUM(x+y);
      }
      if(!strcmp(o,"-")) return VNUM(x-y);
      if(!strcmp(o,"*")) return VNUM(x*y);
      if(!strcmp(o,"/")) return VNUM(y!=0?x/y:0);
      if(!strcmp(o,"%")) return VNUM(y!=0?fmod(x,y):0);
      if(!strcmp(o,"<")) return VBOOL(x<y);
      if(!strcmp(o,"<=")) return VBOOL(x<=y);
      if(!strcmp(o,">")) return VBOOL(x>y);
      if(!strcmp(o,">=")) return VBOOL(x>=y);
      return VNIL();
    }
    case N_QUERY:{ Value a=eval_node(n->kids[0],e); Value b=eval_node(n->kids[1],e);
      if(b.kind==V_FUN||b.kind==V_BUILTIN){ Value args[1]={a}; return call_function(b,args,1); }
      Arr*pair=arr_new(); arr_push(pair,a); arr_push(pair,b); return VARR(pair); }
    case N_COMMIT:{ Value left=eval_node(n->kids[0],e); ledger_append("tuplespace",left); return eval_node(n->kids[1],
e); }
    case N_APPEND:{ Value arrv=eval_node(n->kids[0],e); Value elem=eval_node(n->kids[1],e); if(arrv.kind==V_ARR) arr_p
ush(arrv.arr,elem); return arrv; }
    case N_INDEX:{ Value base=eval_node(n->kids[0],e),idx=eval_node(n->kids[1],e);
      if(base.kind==V_ARR&&idx.kind==V_NUM){ int i=(int)idx.num; if(i>=0&&i<base.arr->n) return base.arr->items[i]; }
      if(base.kind==V_STR&&idx.kind==V_NUM){ int i=(int)idx.num; if(i>=0&&i<(int)strlen(base.str)){ char s[2]={base.st
r[i],0}; return VSTR(s);} }
      return VNIL(); }
    case N_MEMBER:{ Value base=eval_node(n->kids[0],e);
      if(!strcmp(n->str,"length")|||!strcmp(n->str,"len")){ if(base.kind==V_ARR) return VNUM(base.arr->n); if(base.kind
==V_STR) return VNUM(strlen(base.str)); }
      if(!strcmp(n->str,"frozen")) return base;
      if(!strcmp(n->str,"normalize")|||!strcmp(n->str,"normalized")){
        if(base.kind==V_ARR){ double s=0; for(int i=0;i<base.arr->n;i++) s+=base.arr->items[i].num;
          Arr*r=arr_new(); for(int i=0;i<base.arr->n;i++) arr_push(r,VNUM(s!=0?base.arr->items[i].num/s:0)); return VA
RR(r); } }
      return base; }
    case N_SCOPE:{ Value v; v.kind=V_LEDGER; return v; }
    case N_CALL:{
      Node*callee=n->kids[0];
      if(callee->kind==N_MEMBER){
        Value base=eval_node(callee->kids[0],e); const char*m=callee->str;
        int na=n->nkid-1; Value*args=na?xmalloc(na*sizeof(Value)):NULL;
        for(int i=0;i<na;i++) args[i]=eval_node(n->kids[i+1],e);
        if(!strcmp(m,"scan")|||!strcmp(m,"read")){ Arr*out=arr_new();
          if(base.kind==V_ARR&&na>=1) for(int i=0;i<base.arr->n;i++){ Value one[1]={base.arr->items[i]}; if(truthy(cal
l_function(args[0],one,1))) arr_push(out,base.arr->items[i]); }
          return VARR(out); }
        if(!strcmp(m,"append")){ if(base.kind==V_ARR&&na>=1){ arr_push(base.arr,args[0]); return VNUM(base.arr->n-1);}
 }
        if(base.kind==V_LEDGER&&!strcmp(m,"append")){ if(na>=1){ int id=ledger_append("tuplespace",args[0]); return VN
UM(id);} }
        for(int i=0;i<B_NBUILTIN;i++) if(!strcmp(BNAMES[i],m)) return call_builtin(i,args,na);
        return base;
      }
      Value f=eval_node(callee,e); int na=n->nkid-1; Value*args=na?xmalloc(na*sizeof(Value)):NULL;
      for(int i=0;i<na;i++) args[i]=eval_node(n->kids[i+1],e);
      return call_function(f,args,na);
    }
    case N_EACH:{ Value coll=eval_node(n->kids[0],e); Value fn=eval_node(n->kids[1],e);
      if(coll.kind==V_ARR) for(int i=0;i<coll.arr->n;i++){ Value one[1]={coll.arr->items[i]}; call_function(fn,one,1);
 }
      return coll; }
    case N_EXTENDED: return eval_extended(n,e);
    default: return VNIL();
  }
}
static Eval exec_stmt(Node*n,Env*e){
  Eval r={0,VNIL()};
  switch(n->kind){
    case N_RETURN: r.is_return=1; r.val=n->nkid?eval_node(n->kids[0],e):VNIL(); return r;
    case N_PRINT:{ for(int i=0;i<n->nkid;i++){ if(i) printf(" "); print_val(eval_node(n->kids[i],e)); } printf("\n");
return r; }
    case N_IF:{ if(truthy(eval_node(n->kids[0],e))) return exec_block(n->kids[1],e);
      else if(n->nkid>2){ if(n->kids[2]->kind==N_IF) return exec_stmt(n->kids[2],e); else return exec_block(n->kids[2]
,e); } return r; }
    case N_WHILE:{ while(truthy(eval_node(n->kids[0],e))){ Eval x=exec_block(n->kids[1],e); if(x.is_return) return x;
} return r; }
    case N_FOR:{ Value coll=eval_node(n->kids[0],e); if(coll.kind==V_ARR){ Env*l=env_new(e);
        for(int i=0;i<coll.arr->n;i++){ env_bind(l,n->str,coll.arr->items[i]); Eval x=exec_block(n->kids[1],l); if(x.i
s_return) return x; } } return r; }
```

```
    case N_BLOCK: return exec_block(n,e);
    default: eval_node(n,e); return r;
  }
}
static Eval exec_block(Node*blk,Env*e){ Eval r={0,VNIL()}; for(int i=0;i<blk->nkid;i++){ r=exec_stmt(blk->kids[i],e); if(r.is_return) return r; } return r; }

static Value call_builtin(int idx,Value*args,int nargs){
  switch(idx){
    case B_SOFTMAX:{ int n; double*z=val_to_num_arr(nargs?args[0]:VNIL(),&n); if(n==0) return VNIL();
      double*o=xmalloc(n*sizeof(double)); softmax_vec(z,n,o); Value v=num_arr_to_val(o,n); free(z); free(o); return v; }
    case B_ENTROPY:{ int n; double*q=val_to_num_arr(nargs?args[0]:VNIL(),&n); double h=shannon_entropy(q,n); free(q); return VNUM(h); }
    case B_UNKNOWN_PRIOR:{ int V=nargs?(int)args[0].num:0; if(V<=0) return VNIL(); double*o=xmalloc(V*sizeof(double)); for(int i=0;i<V;i++) o[i]=1.0/V; Value v=num_arr_to_val(o,V); free(o); return v; }
    case B_UPDATE:{ int n,m; double*pri=val_to_num_arr(args[0],&n); double*ev=val_to_num_arr(args[1],&m); double lr=nargs>2?args[2].num:0.5;
      if(n==0){ free(pri); free(ev); return VNIL(); } double*o=xmalloc(n*sizeof(double)); double s=0;
      for(int i=0;i<n;i++){ o[i]=(1-lr)*pri[i]+lr*(i<m?ev[i]:0); s+=o[i]; } for(int i=0;i<n;i++) o[i]/=(s!=0?s:1);
      Value v=num_arr_to_val(o,n); free(pri); free(ev); free(o); return v; }
    case B_MANIFOLD_EMBED:{ int n; double*x=val_to_num_arr(nargs?args[0]:VNIL(),&n); if(n==0) return VNIL();
      double nrm=0; for(int i=0;i<n;i++) nrm+=x[i]*x[i]; nrm=sqrt(nrm)+1e-12; double*o=xmalloc(n*sizeof(double));
      for(int i=0;i<n;i++) o[i]=x[i]/nrm; Value v=num_arr_to_val(o,n); free(x); free(o); return v; }
    case B_COGNITIVE:{
      int V; double*z=val_to_num_arr(args[0],&V); if(V==0) return VNIL();
      int M=0; double**qm=NULL,**phim=NULL,*beta=NULL;
      if(nargs>1&&args[1].kind==V_ARR){ M=args[1].arr->n; qm=xmalloc((M>0?M:1)*sizeof(double*)); phim=xmalloc((M>0?M:1)*sizeof(double*));
        for(int m=0;m<M;m++){ Value pair=args[1].arr->items[m]; int a,b;
          if(pair.kind==V_ARR&&pair.arr->n>=2){ qm[m]=val_to_num_arr(pair.arr->items[0],&a); phim[m]=val_to_num_arr(pair.arr->items[1],&b);
            if(a<V||b<V){ double*Q=xmalloc(V*sizeof(double)),*PH=xmalloc(V*sizeof(double)); for(int i=0;i<V;i++){ Q[i]=i<a?qm[m][i]:1.0/V; PH[i]=i<b?phim[m][i]:0; } free(qm[m]); free(phim[m]); qm[m]=Q; phim[m]=PH; } }
          else { qm[m]=xmalloc(V*sizeof(double)); phim[m]=xmalloc(V*sizeof(double)); for(int i=0;i<V;i++){ qm[m][i]=1.0/V; phim[m][i]=0; } } } }
      beta=xmalloc((M>0?M:1)*sizeof(double));
      if(nargs>2&&args[2].kind==V_ARR) for(int m=0;m<M;m++) beta[m]=m<args[2].arr->n?args[2].arr->items[m].num:1.0; else for(int m=0;m<M;m++) beta[m]=1.0;
      double*q_hat=xmalloc(V*sizeof(double)),md; cognitive_system(z,V,qm,phim,beta,M,q_hat,&md); G_LAST_MAXDIFF=md;
      if(!G_LAST_A) G_LAST_A=arr_new(); G_LAST_A->n=0; double*a=xmalloc(V*sizeof(double)); softmax_vec(z,V,a);
      for(int i=0;i<V;i++) arr_push(G_LAST_A,VNUM(a[i]));
      Value out=num_arr_to_val(q_hat,V); ledger_append("tuplespace",out);
      free(z); free(beta); free(q_hat); free(a); for(int m=0;m<M;m++){ free(qm[m]); free(phim[m]); } free(qm); free(phim);
      return out; }
    case B_DIST:{ int n; double*z=val_to_num_arr(nargs?args[0]:VNIL(),&n); if(n==0) return VNIL();
      double s=0; for(int i=0;i<n;i++) s+=z[i]; double*o=xmalloc(n*sizeof(double));
      for(int i=0;i<n;i++) o[i]=s>0?z[i]/s:1.0/n; Value v=num_arr_to_val(o,n); free(z); free(o); return v; }
    case B_ZEROS_OF:{ int n; double*q=val_to_num_arr(nargs?args[0]:VNIL(),&n); Arr*z=arr_new(); for(int i=0;i<n;i++) if(q[i]==0.0) arr_push(z,VNUM(i)); free(q); return VARR(z); }
    case B_MAXDIFF:{ if(nargs>=2&&args[0].kind==V_ARR&&args[1].kind==V_ARR){ int na,nb; double*A=val_to_num_arr(args[0],&na); double*B=val_to_num_arr(args[1],&nb);
        int V=na<nb?na:nb; double md=0; for(int i=0;i<V;i++){ double d=fabs(A[i]-B[i]); if(d>md) md=d; } free(A); free(B); return VNUM(md); }
      return VNUM(G_LAST_MAXDIFF); }
    case B_LAST_A: return G_LAST_A?VARR(G_LAST_A):VNIL();
    case B_LEDGER:{ Arr*a=arr_new(); for(int i=0;i<LEDGER.n;i++) arr_push(a,LEDGER.facts[i].v); return VARR(a); }
    case B_LEN:{ Value v=nargs?args[0]:VNIL(); if(v.kind==V_ARR) return VNUM(v.arr->n); if(v.kind==V_STR) return VNUM(strlen(v.str)); return VNUM(0); }
    case B_SQRT: return VNUM(sqrt(nargs?args[0].num:0));
    case B_LOG: return VNUM(log(nargs?args[0].num:1));
    case B_EXP: return VNUM(exp(nargs?args[0].num:0));
    case B_ABS: return VNUM(fabs(nargs?args[0].num:0));
    case B_F5:{ char b[64]; snprintf(b,64,"%.5f",nargs?args[0].num:0); return VSTR(b); }
    case B_SCI:{ char b[64]; snprintf(b,64,"%.2e",nargs?args[0].num:0); return VSTR(b); }
    case B_CHAR_AT:{ Value s=args[0]; int i=(int)args[1].num; if(s.kind==V_STR&&i>=0&&i<(int)strlen(s.str)){ char c[2]={s.str[i],0}; return VSTR(c);} return VSTR(""); }
    case B_STRLEN: return VNUM(nargs&&args[0].kind==V_STR?strlen(args[0].str):0);
    case B_SIN: return VNUM(sin(nargs?args[0].num:0));
    case B_COS: return VNUM(cos(nargs?args[0].num:0));
    case B_POW: return VNUM(pow(nargs?args[0].num:0, nargs>1?args[1].num:0));
    case B_FLOOR: return VNUM(floor(nargs?args[0].num:0));
    case B_MAX: return VNUM(nargs>1?(args[0].num>args[1].num?args[0].num:args[1].num):(nargs?args[0].num:0));
    case B_MIN: return VNUM(nargs>1?(args[0].num<args[1].num?args[0].num:args[1].num):(nargs?args[0].num:0));
    case B_MEAN:{ int n; double*x=val_to_num_arr(nargs?args[0]:VNIL(),&n); if(n==0){free(x);return VNUM(0);} double s=0; for(int i=0;i<n;i++) s+=x[i]; free(x); return VNUM(s/n); }
    case B_STDEV:{ int n; double*x=val_to_num_arr(nargs?args[0]:VNIL(),&n); if(n<2){free(x);return VNUM(0);} double m=0; for(int i=0;i<n;i++) m+=x[i]; m/=n; double v=0; for(int i=0;i<n;i++){ double d=x[i]-m; v+=d*d; } free(x); return VNUM(sqrt(v/n)); }
    case B_CORREL:{ /* Pearson correlation of two equal-length numeric arrays */
```

```c
      if(nargs<2||args[0].kind!=V_ARR||args[1].kind!=V_ARR) return VNUM(0);
      int na,nb; double*A=val_to_num_arr(args[0],&na); double*B=val_to_num_arr(args[1],&nb); int n=na<nb?na:nb;
      if(n==0){ free(A); free(B); return VNUM(0); }
      double ma=0,mb=0; for(int i=0;i<n;i++){ ma+=A[i]; mb+=B[i]; } ma/=n; mb/=n;
      double sab=0,saa=0,sbb=0; for(int i=0;i<n;i++){ double da=A[i]-ma,db=B[i]-mb; sab+=da*db; saa+=da*da; sbb+=db*db; }
      double den=sqrt(saa*sbb)+1e-300; double r=sab/den; free(A); free(B); return VNUM(r); }
    default: return VNIL();
  }
}
static Env*GLOBAL=NULL;
static void register_func(Node*fn,Env*e){ Value v; v.kind=V_FUN; v.fn=fn; v.clo=e; env_bind(e,fn->str,v); }
static void exec_reviser(Node*rev,Env*e){
  for(int i=0;i<rev->nkid;i++){ Node*k=rev->kids[i]; if(k->kind==N_RULE){ const char*head=k->kids[0]->str; grammar_commit(k->str,head,k->kids[1],k->kids[2]); } }
  for(int i=0;i<rev->nkid;i++){ Node*k=rev->kids[i]; if(k->kind!=N_RULE){ if(k->kind==N_FUNC) register_func(k,e); else exec_stmt(k,e); } }
}
static void exec_decl(Node*d,Env*e){
  switch(d->kind){
    case N_FUNC: register_func(d,e); break;
    case N_REVISER: exec_reviser(d,e); break;
    case N_OMEGABLOCK: for(int i=0;i<d->kids[0]->nkid;i++) exec_decl(d->kids[0]->kids[i],e); break;
    case N_ASPERAL: if(d->nkid&&d->kids[d->nkid-1]->kind==N_BLOCK) for(int i=0;i<d->kids[d->nkid-1]->nkid;i++) exec_decl(d->kids[d->nkid-1]->kids[i],e); break;
    case N_TYPEDEF: break;
    default: exec_stmt(d,e); break;
  }
}
static void run_program(Node*prog){
  GLOBAL=env_new(NULL);
  for(int i=0;i<B_NBUILTIN;i++){ Value v; v.kind=V_BUILTIN; v.builtin=i; env_bind(GLOBAL,BNAMES[i],v); }
  for(int i=0;i<prog->nkid;i++) if(prog->kids[i]->kind==N_FUNC) register_func(prog->kids[i],GLOBAL);
  for(int i=0;i<prog->nkid;i++) if(prog->kids[i]->kind==N_REVISER) exec_reviser(prog->kids[i],GLOBAL);
  for(int i=0;i<prog->nkid;i++){ Node*d=prog->kids[i]; if(d->kind==N_FUNC||d->kind==N_REVISER) continue; exec_decl(d,GLOBAL); }
}

/* ---------------------------------------------------------- dumps & CLI -- */
static void dump_tokens(Token*t){
  for(size_t i=0;;i++){ Token*k=&t[i]; printf("%-12s",tok_name(k->kind));
    if(k->kind==T_INT||k->kind==T_FLOAT) printf(" %g",k->num); else if(k->lex) printf(" %s",k->lex);
    printf("\n"); if(k->kind==T_EOF) break; }
}
static char*read_file(const char*path){ FILE*f=fopen(path,"rb"); if(!f){ fprintf(stderr,"cannot open %s\n",path); exit(1); }
  fseek(f,0,SEEK_END); long n=ftell(f); fseek(f,0,SEEK_SET); char*b=xmalloc(n+1); size_t r=fread(b,1,n,f); b[r]=0; fclose(f); return b; }
static Node*parse_source(const char*src,Token**toks_out){
  Lexer L; memset(&L,0,sizeof L); L.src=src; L.len=strlen(src); L.line=1; lex_all(&L);
  if(toks_out) *toks_out=L.toks; Parser P; P.t=L.toks; P.i=0; P.n=L.ntok; return parse_program(&P);
}
int main(int argc,char**argv){
  if(argc<3){ fprintf(stderr,"usage: %s <run|tokens|ast> file.bada\n",argv[0]); return 1; }
  const char*cmd=argv[1]; const char*path=argv[2]; char*src=read_file(path);
  if(!strcmp(cmd,"tokens")){ Token*t; parse_source(src,&t); dump_tokens(t); return 0; }
  if(!strcmp(cmd,"run")){ Node*prog=parse_source(src,NULL); run_program(prog); return 0; }
  fprintf(stderr,"unknown command: %s\n",cmd); return 1;
}
```

2. `predictor.bada`

The Unknown-Prior analysis application, written in Bada. Encodes a hypothesis as a distribution with structural zeros, proves zero-preservation and |psi|^2 = a, runs the reviser loop so entropy strictly decreases, and performs the honest zero-structure probe (multiplication preserves zeros; addition does not).

```
# ============================================================================
# predictor.bada -- The Unknown-Prior Analysis Engine
#
# An inference console written in Bada, running on the Unknown-Prior Engine.
# It does NOT claim to solve the Millennium Problems or to "predict the unknown."
# It does something real and checkable:
#
#   1. encodes a hypothesis as a base distribution with structural zeros
#      (ruled-out outcomes = "confident zeros" = check symbols);
#   2. runs the entropy-phase core  psi = sqrt(a) * exp(i*theta);
#      proves the two provable properties at runtime:
#         (i)  zero preservation     -- a confident zero is never resurrected
#         (ii) |psi|^2 == a          -- the phase only reweights, never destroys
#   3. earns certainty honestly: a max-entropy prior is sharpened by evidence
#      inside an @reviser learning loop, and entropy strictly decreases;
#   4. analyzes the Millennium / zeta question the ONLY way the reports actually
#      support: by testing whether a combination operation preserves a zero
#      pattern. Addition destroys zeros; multiplication preserves them. This is
#      the verifiable core of the Yamaguchi reports (Document 5), reproduced
#      numerically -- not a proof of RH.
# ============================================================================

Omega::DATABASE[tuplespace] {

  print("================================================================")
  print("  UNKNOWN-PRIOR ANALYSIS ENGINE  (Bada / honest inference core)")
  print("================================================================")
  print("")

  # ============================================================================
  # PART 1 -- the cognitive core: encode, phase, prove the two properties
  # ============================================================================
  # logits over V = 8 hypotheses. Slots 3 and 6 are driven to -800 so softmax
  # rules them out exactly: these are the structural zeros / check symbols.
  z := [2.0, 1.0, 0.5, -800.0, 0.1, 1.5, -800.0, 0.3]

  # one informative module + one uninformed module (constant H -> cancels)
  evidence := [0.30, 0.20, 0.10, 0.00, 0.05, 0.20, 0.00, 0.15]
  phi_e    := [0.4, -0.2, 0.1, 0.0, 0.3, -0.5, 0.0, 0.2]
  flat     := [0.125,0.125,0.125,0.125,0.125,0.125,0.125,0.125]
  phi_0    := [0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0]

  modules := [ [evidence, phi_e], [flat, phi_0] ]
  betas   := [0.7, 0.5]

  a     := softmax(z)
  q_hat := cognitive_system(z, modules, betas)

  print("[core] vocabulary V        =", len(a))
  print("[core] base a              =", a)
  print("[core] posterior q_hat     =", q_hat)
  print("")
  print("[proof i ] base zeros      =", zeros_of(a))
  print("[proof i ] posterior zeros =", zeros_of(q_hat), "(preserved)")
  print("[proof ii] |psi|^2 == a    =", sci(maxdiff()), "(machine zero)")
  print("")

  # ============================================================================
  # PART 2 -- earn certainty: @reviser learning loop, entropy must decrease
  # ============================================================================
  print("----------------------------------------------------------------")
  print("  reviser loop: an honest prior sharpens as evidence arrives")
  print("----------------------------------------------------------------")

  prior := unknown_prior(8)
  print("step 0 (max entropy / Jaynes) entropy =", f5(entropy(prior)))

  ev1 := [0.40, 0.14, 0.08, 0.00, 0.05, 0.23, 0.00, 0.10]
  ev2 := [0.55, 0.10, 0.05, 0.00, 0.03, 0.22, 0.00, 0.05]

  @reviser {
    prior := update(prior, ev1)
    prior >> tuplespace
```

```
}
print("step 1 (evidence frame 1)     entropy =", f5(entropy(prior)))

@reviser {
  prior := update(prior, ev2)
  prior >> tuplespace
}
print("step 2 (evidence frame 2)     entropy =", f5(entropy(prior)))
print("ledger facts committed (audit trail)  =", len(ledger()))
print("")
print("(entropy strictly decreases: the engine earns its certainty)")
print("")

# ============================================================================
# PART 3 -- millennium / zeta STRUCTURE analysis  (honest, not a proof)
# ============================================================================
# The reports' one verifiable claim (Document 5): a non-zero factor times a
# zero pattern preserves its zeros (multiplication); a sum does not (addition).
# We test it directly. zero_pattern marks where a "zeta-like" object vanishes.
print("-----------------------------------------------------------------")
print("  zero-structure probe: does an operation preserve the zeros?")
print("-----------------------------------------------------------------")

# a stand-in zeta-like object over 8 sample points. A 0 entry IS a zero of
# the object (it vanishes there); nonzero entries are its ordinary values.
zeta := [2, 1, 0, 3, 1, 0, 2, 0]

# an arbitrary factor field that is itself NEVER zero anywhere
factor := [3, 1, 2, 4, 1, 5, 2, 3]

# a noise field we will ADD (generically nonzero, masks the zeros)
noise  := [1, 2, 1, 3, 2, 1, 4, 2]

# MULTIPLICATIVE combine: zero(zeta) * anything = 0  -> zeros survive
mult := [ zeta[0]*factor[0], zeta[1]*factor[1], zeta[2]*factor[2], zeta[3]*factor[3],
          zeta[4]*factor[4], zeta[5]*factor[5], zeta[6]*factor[6], zeta[7]*factor[7] ]

# ADDITIVE combine: 0(zeta) + nonzero(noise) != 0  -> zeros destroyed
addt := [ zeta[0]+noise[0], zeta[1]+noise[1], zeta[2]+noise[2], zeta[3]+noise[3],
          zeta[4]+noise[4], zeta[5]+noise[5], zeta[6]+noise[6], zeta[7]+noise[7] ]

print("zeta-like object    :", zeta,  "-> zeros at", zeros_of(zeta))
print("x factor (mult)     :", mult,  "-> zeros at", zeros_of(mult))
print("+ noise  (add)      :", addt,  "-> zeros at", zeros_of(addt))
print("")

# fraction of the original object's zeros that survive in the combined object
def zeros_kept |orig, combined| {
  kept  := [0]
  total := [0]
  idx   := [0]
  while (idx[len(idx) - 1] < len(orig)) {
    i := idx[len(idx) - 1]
    if (orig[i] == 0) {
      total <- (total[len(total) - 1] + 1)
      if (combined[i] == 0) { kept <- (kept[len(kept) - 1] + 1) }
    }
    idx <- (i + 1)
  }
  t := total[len(total) - 1]
  if (t == 0) { return 0 }
  return kept[len(kept) - 1] / t
}

mult_pres := zeros_kept(zeta, mult)
add_pres  := zeros_kept(zeta, addt)

print("multiplication: zeros preserved =", f5(mult_pres), "(1.0 = all kept)")
print("addition:       zeros preserved =", f5(add_pres),  "(0.0 = all lost)")
print("")

# commit the verdict to the append-only ledger as an auditable fact
mult >> tuplespace
addt >> tuplespace

print("VERDICT (verifiable):")
print("  Only multiplication by a non-zero factor preserves the zero")
print("  structure. Addition does not. This matches Document 5 and is the")
print("  ONLY claim about zeta the reports actually support numerically.")
print("")
print("NOT CLAIMED: no proof of the Riemann Hypothesis or any Millennium")
```

```
  print("  Problem follows. The zeta = beta/log x identities in the manuscripts")
  print("  are not used; no engine can produce an unknown closed form.")
  print("================================================================")
}
```

3. engine.html

Interactive Unknown-Prior console. A faithful JavaScript port of the engine numerics so it runs in any browser with no server. Ink-black ground, gold / vermilion / cyan accents.

```
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
<title>[redacted] · Unknown-Prior Analysis Engine</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Shippori+Mincho:wght@500;600;800&family=Share+Tech+Mono&family=Orbitron:wght@500;700&display=swap" rel="stylesheet">
<style>
  :root{
    --ink:#0a0b0f;          /* ink-black ground */
    --ink-2:#12141c;
    --panel:#161922;
    --line:#262b38;
    --gold:#d9a441;         /* gold accent */
    --vermilion:#e0533a;    /* vermilion */
    --cyan:#4fd6d6;         /* cyan */
    --jade:#6fcf97;         /* truth-green for verified facts */
    --fog:#9aa2b3;
    --fog-2:#5c6373;
    --paper:#eceae2;
  }
  *{box-sizing:border-box;margin:0;padding:0}
  html,body{height:100%}
  body{
    background:
      radial-gradient(1200px 500px at 50% -10%, #1a1d28 0%, transparent 60%),
      var(--ink);
    color:var(--paper);
    font-family:"Shippori Mincho",serif;
    -webkit-font-smoothing:antialiased;
    overflow-x:hidden;
  }
  .wrap{max-width:760px;margin:0 auto;padding:22px 16px 120px;min-height:100%}

  /* ---- header: the thesis ---- */
  header{padding:14px 4px 20px;border-bottom:1px solid var(--line);margin-bottom:18px}
  .kicker{font-family:"Share Tech Mono",monospace;font-size:11px;letter-spacing:.34em;
    color:var(--gold);text-transform:uppercase;margin-bottom:10px}
  h1{font-weight:800;font-size:26px;line-height:1.25;letter-spacing:.01em}
  h1 .en{display:block;font-family:"Orbitron",sans-serif;font-weight:500;font-size:13px;
    letter-spacing:.18em;color:var(--fog);margin-top:8px}
  .law{margin-top:16px;padding:12px 14px;background:var(--ink-2);border:1px solid var(--line);
    border-left:2px solid var(--cyan);border-radius:4px}
  .law .eq{font-family:"Share Tech Mono",monospace;font-size:13px;color:var(--cyan);line-height:1.7}
  .law .note{font-size:12.5px;color:var(--fog);margin-top:6px;line-height:1.6}

  /* ---- honesty banner ---- */
  .honesty{margin:16px 0 4px;padding:11px 14px;border:1px dashed var(--vermilion);
    border-radius:4px;background:rgba(224,83,58,.06)}
  .honesty b{color:var(--vermilion);font-family:"Share Tech Mono",monospace;font-size:11px;
    letter-spacing:.18em;text-transform:uppercase}
  .honesty p{font-size:12.5px;color:var(--fog);line-height:1.65;margin-top:5px}

  /* ---- run controls ---- */
  .panel{margin-top:20px;background:var(--panel);border:1px solid var(--line);border-radius:8px;
    overflow:hidden}
  .panel .bar{display:flex;align-items:center;gap:8px;padding:10px 13px;border-bottom:1px solid var(--line);
    background:var(--ink-2)}
  .dot{width:9px;height:9px;border-radius:50%;background:var(--fog-2)}
  .dot.live{background:var(--jade);box-shadow:0 0 8px var(--jade)}
  .bar .title{font-family:"Share Tech Mono",monospace;font-size:11px;letter-spacing:.2em;
    color:var(--fog);text-transform:uppercase;margin-left:2px}
  .bar .spacer{flex:1}
  .bar .v{font-family:"Share Tech Mono",monospace;font-size:10px;color:var(--fog-2)}

  .field{padding:14px 13px 4px}
  .field label{display:block;font-family:"Share Tech Mono",monospace;font-size:10.5px;
    letter-spacing:.16em;color:var(--gold);text-transform:uppercase;margin-bottom:7px}
  .hyp{display:flex;flex-wrap:wrap;gap:7px}
  .hyp input{flex:1 1 120px;min-width:0;background:var(--ink);border:1px solid var(--line);
    color:var(--paper);font-family:"Share Tech Mono",monospace;font-size:13px;
```

```
    padding:9px 10px;border-radius:5px}
  .hyp input:focus{outline:none;border-color:var(--cyan)}
  .hint{font-size:11.5px;color:var(--fog-2);margin:8px 2px 0;line-height:1.55}

  .ruleout{padding:6px 13px 14px}
  .ruleout .chips{display:flex;flex-wrap:wrap;gap:6px;margin-top:7px}
  .chip{font-family:"Share Tech Mono",monospace;font-size:12px;padding:6px 11px;border-radius:20px;
    border:1px solid var(--line);background:var(--ink);color:var(--fog);cursor:pointer;
    user-select:none;transition:all .12s}
  .chip.on{border-color:var(--vermilion);color:var(--vermilion);background:rgba(224,83,58,.1)}
  .chip:active{transform:scale(.94)}

  .actions{display:flex;gap:9px;padding:4px 13px 15px}
  button.run{flex:1;font-family:"Orbitron",sans-serif;font-weight:700;font-size:13px;
    letter-spacing:.08em;padding:13px;border:none;border-radius:6px;cursor:pointer;
    color:var(--ink);background:linear-gradient(180deg,var(--gold),#b9852c);
    box-shadow:0 2px 0 #7a5717}
  button.run:active{transform:translateY(1px);box-shadow:0 1px 0 #7a5717}
  button.ghost{font-family:"Share Tech Mono",monospace;font-size:12px;letter-spacing:.1em;
    padding:13px 16px;border:1px solid var(--line);border-radius:6px;background:var(--ink);
    color:var(--fog);cursor:pointer}
  button.ghost:active{transform:translateY(1px)}

  /* ---- console output ---- */
  .console{margin-top:18px;background:#070809;border:1px solid var(--line);border-radius:8px;
    font-family:"Share Tech Mono",monospace;font-size:12.5px;line-height:1.85;
    padding:14px 14px 18px;min-height:80px;white-space:pre-wrap;word-break:break-word;
    overflow-x:auto}
  .console .ln{display:block}
  .c-head{color:var(--gold)}
  .c-rule{color:var(--fog-2)}
  .c-key{color:var(--fog)}
  .c-ok{color:var(--jade)}
  .c-zero{color:var(--cyan)}
  .c-warn{color:var(--vermilion)}
  .c-num{color:var(--paper)}
  .cursor{display:inline-block;width:8px;height:15px;background:var(--gold);
    vertical-align:-2px;animation:blink 1s steps(1) infinite}
  @keyframes blink{50%{opacity:0}}

  /* ---- verdict cards ---- */
  .cards{display:grid;grid-template-columns:1fr 1fr;gap:10px;margin-top:14px}
  .card{background:var(--panel);border:1px solid var(--line);border-radius:8px;padding:13px}
  .card .op{font-family:"Share Tech Mono",monospace;font-size:11px;letter-spacing:.14em;
    text-transform:uppercase;color:var(--fog)}
  .card .big{font-family:"Orbitron",sans-serif;font-weight:700;font-size:28px;margin:6px 0 2px}
  .card.keep .big{color:var(--jade)} .card.lose .big{color:var(--vermilion)}
  .card .sub{font-size:11.5px;color:var(--fog-2);line-height:1.5}

  footer{margin-top:26px;padding:14px 4px 0;border-top:1px solid var(--line);
    font-size:11.5px;color:var(--fog-2);line-height:1.7}
  footer code{font-family:"Share Tech Mono",monospace;color:var(--gold)}
  @media (max-width:430px){ h1{font-size:22px} .cards{grid-template-columns:1fr} }
  @media (prefers-reduced-motion:reduce){ .cursor{animation:none} * {transition:none!important} }
</style>
</head>
<body>
<div class="wrap">

  <header>
    <div class="kicker">Bada · Unknown-Prior Engine</div>
    <h1>[redacted]
      <span class="en">UNKNOWN-PRIOR ANALYSIS ENGINE</span>
    </h1>
    <div class="law">
      <div class="eq">ψ[redacted] = √a[redacted] · e^{iθ[redacted]}  →  |ψ[redacted]|² = a[redacted]  (exact, 5.55e-17)</div>
      <div class="note">[redacted]</div>
    </div>
  </header>

  <div class="honesty">
    <b>[redacted] / what this is</b>
    <p>[redacted] |ψ|²=a [redacted]<b>[redacted]</b>[redacted]
[redacted]</p>
  </div>

  <div class="panel">
    <div class="bar">
      <span class="dot" id="dot"></span>
      <span class="title">cognitive core</span>
      <span class="spacer"></span>
```

```
      <span class="v">V=8 · complex.h</span>
    </div>

    <div class="field">
      <label>████████ (8 hypotheses · logits z)</label>
      <div class="hyp" id="logits"></div>
      <div class="hint">████████████████████████ softmax ████████████████████████</div>
    </div>

    <div class="ruleout">
      <label style="font-family:'Share Tech Mono',monospace;font-size:10.5px;letter-spacing:.16em;color:var(--vermilion);text-transform:uppercase">██████ / ruled-out (check symbols)</label>
      <div class="chips" id="chips"></div>
    </div>

    <div class="actions">
      <button class="run" id="runBtn">██████ · RUN ANALYSIS</button>
      <button class="ghost" id="resetBtn">reset</button>
    </div>
  </div>

  <div class="console" id="out"><span class="ln c-rule">engine idle. ████████ RUN ██████████</span><span class="cursor"></span></div>

  <div class="cards" id="cards" style="display:none">
    <div class="card keep" id="cardMult">
      <div class="op">× multiplication</div>
      <div class="big" id="multVal">—</div>
      <div class="sub">██████ · zeros preserved</div>
    </div>
    <div class="card lose" id="cardAdd">
      <div class="op">+ addition</div>
      <div class="big" id="addVal">—</div>
      <div class="sub">██████ · zeros destroyed</div>
    </div>
  </div>

  <footer>
    ████████████ Bada ███████ (<code>bada.c</code>) ████████████████████ JS ████████<code>softmax</code> / <code>cognitive_system</code> / <code>@reviser</code> / append-only <code>tuplespace</code>■
    <br>████████ RH■NRH ████████████████████████████
  </footer>
</div>

<script>
"use strict";
/* ========================================================================
 * Faithful JS port of the Bada Unknown-Prior Engine numerics.
 * Mirrors bada.c: softmax (with confident-zero rule), shannon entropy,
 * cognitive_system (psi = sqrt(a)*exp(i theta), |psi|^2 = a), update,
 * zeros_of, maxdiff, and an append-only ledger.
 * ======================================================================== */

const RULE_OUT = -800;          // logit that denotes a confident zero
const ledger = [];              // append-only tuplespace

function softmax(z){
  let mx = -Infinity;
  for(const v of z) if(v > mx) mx = v;
  const out = new Array(z.length);
  let s = 0;
  for(let i=0;i<z.length;i++){
    out[i] = (z[i] === -Infinity || (mx - z[i]) > 30) ? 0 : Math.exp(z[i]-mx);
    s += out[i];
  }
  for(let i=0;i<z.length;i++) out[i] /= s;
  return out;
}
function entropy(q){
  let h = 0;
  for(const x of q) if(x > 1e-12) h -= x*Math.log(x);
  return h;
}
function zerosOf(d){
  const z = [];
  for(let i=0;i<d.length;i++) if(d[i] === 0) z.push(i);
  return z;
}
function maxdiff(a,b){
  let m = 0; const n = Math.min(a.length,b.length);
  for(let i=0;i<n;i++){ const d = Math.abs(a[i]-b[i]); if(d>m) m=d; }
```

```
  return m;
}
function update(prior, ev, lr=0.5){
  const n = prior.length, out = new Array(n); let s=0;
  for(let i=0;i<n;i++){ out[i] = (1-lr)*prior[i] + lr*(i<ev.length?ev[i]:0); s += out[i]; }
  for(let i=0;i<n;i++) out[i] /= (s||1);
  return out;
}
/* cognitive_system over base logits z, modules [[q_m, phi_m]...], gains beta_m.
   Returns {q_hat, a, md}. |psi|^2 = a exactly because |e^{i theta}| = 1. */
function cognitiveSystem(z, modules, betas){
  const a = softmax(z), V = a.length;
  const theta = new Array(V).fill(0);
  for(let m=0;m<modules.length;m++){
    const [qm, phim] = modules[m];
    let H = 0; for(const x of qm) if(x>1e-12) H -= x*Math.log(x);
    const beta = betas[m] ?? 1;
    for(let i=0;i<V;i++) theta[i] += beta*H*(phim[i] ?? 0);
  }
  const mag2 = new Array(V); let sum=0;
  for(let i=0;i<V;i++){
    const re = Math.sqrt(a[i])*Math.cos(theta[i]);
    const im = Math.sqrt(a[i])*Math.sin(theta[i]);
    mag2[i] = re*re + im*im;          // = a[i]
    sum += mag2[i];
  }
  const q_hat = mag2.map(x=>x/sum);
  const md = maxdiff(q_hat, a);
  ledger.push({region:"tuplespace", v:q_hat.slice()});
  return {q_hat, a, md};
}

/* ---------------- UI state ---------------- */
const DEFAULT_Z = [2.0, 1.0, 0.5, RULE_OUT, 0.1, 1.5, RULE_OUT, 0.3];
let z = DEFAULT_Z.slice();
let ruledOut = new Set([3,6]);

const logitsEl = document.getElementById("logits");
const chipsEl  = document.getElementById("chips");
const outEl    = document.getElementById("out");
const dotEl    = document.getElementById("dot");
const cardsEl  = document.getElementById("cards");

function buildInputs(){
  logitsEl.innerHTML = "";
  chipsEl.innerHTML = "";
  for(let i=0;i<8;i++){
    const inp = document.createElement("input");
    inp.type = "text";
    inp.inputMode = "decimal";
    inp.value = ruledOut.has(i) ? "ruled-out" : String(z[i]);
    inp.disabled = ruledOut.has(i);
    inp.dataset.i = i;
    inp.addEventListener("change", e=>{
      const k = +e.target.dataset.i;
      const val = parseFloat(e.target.value);
      if(!Number.isNaN(val)) z[k] = val;
    });
    logitsEl.appendChild(inp);

    const chip = document.createElement("div");
    chip.className = "chip" + (ruledOut.has(i) ? " on" : "");
    chip.textContent = "h" + i;
    chip.addEventListener("click", ()=>{
      if(ruledOut.has(i)){ ruledOut.delete(i); z[i] = DEFAULT_Z[i] === RULE_OUT ? 0.0 : DEFAULT_Z[i]; }
      else { ruledOut.add(i); z[i] = RULE_OUT; }
      buildInputs();
    });
    chipsEl.appendChild(chip);
  }
}

function line(cls, txt){ return `<span class="ln ${cls}">${txt}</span>`; }
const f5 = x => x.toFixed(5);
const sci = x => x.toExponential(2);

async function typeOut(lines){
  outEl.innerHTML = "";
  for(const html of lines){
    outEl.insertAdjacentHTML("beforeend", html);
    outEl.scrollTop = outEl.scrollHeight;
```

```
    await new Promise(r=>setTimeout(r, 38));
  }
  outEl.insertAdjacentHTML("beforeend", `<span class="cursor"></span>`);
}

async function run(){
  dotEl.classList.add("live");
  ledger.length = 0;

  /* effective z: ruled-out slots forced to RULE_OUT */
  const Z = z.map((v,i)=> ruledOut.has(i) ? RULE_OUT : v);

  /* one informative + one uninformed module (constant H -> cancels) */
  const evidence = [0.30,0.20,0.10,0.00,0.05,0.20,0.00,0.15];
  const phi_e    = [0.4,-0.2,0.1,0.0,0.3,-0.5,0.0,0.2];
  const flat     = [0.125,0.125,0.125,0.125,0.125,0.125,0.125,0.125];
  const phi_0    = [0,0,0,0,0,0,0,0];
  const modules  = [[evidence,phi_e],[flat,phi_0]];
  const betas    = [0.7,0.5];

  const a = softmax(Z);
  const {q_hat, md} = cognitiveSystem(Z, modules, betas);

  /* reviser learning loop: entropy must strictly decrease */
  let prior = new Array(8).fill(1/8);
  const e0 = entropy(prior);
  prior = update(prior,[0.40,0.14,0.08,0.00,0.05,0.23,0.00,0.10]);
  ledger.push({region:"tuplespace", v:prior.slice()});
  const e1 = entropy(prior);
  prior = update(prior,[0.55,0.10,0.05,0.00,0.03,0.22,0.00,0.05]);
  ledger.push({region:"tuplespace", v:prior.slice()});
  const e2 = entropy(prior);

  /* zero-structure probe (Document 5): mult preserves zeros, add destroys */
  const zeta   = [2,1,0,3,1,0,2,0];
  const factor = [3,1,2,4,1,5,2,3];
  const noise  = [1,2,1,3,2,1,4,2];
  const mult = zeta.map((v,i)=> v*factor[i]);
  const addt = zeta.map((v,i)=> v+noise[i]);
  const origZeros = zerosOf(zeta);
  const keptMult = origZeros.filter(i=> mult[i]===0).length / (origZeros.length||1);
  const keptAdd  = origZeros.filter(i=> addt[i]===0).length / (origZeros.length||1);

  const fmt = arr => "[" + arr.map(x=> Number.isInteger(x)? x : +x.toFixed(5)).join(", ") + "]";

  const lines = [
    line("c-head","■■ PART 1 · cognitive core ■■"),
    line("c-key","V              = " + a.length),
    line("c-num","base a         = " + fmt(a)),
    line("c-num","posterior q■    = " + fmt(q_hat)),
    line("c-zero","zeros(a)       = " + fmt(zerosOf(a))),
    line("c-zero","zeros(q■)       = " + fmt(zerosOf(q_hat)) + "   ← preserved"),
    line("c-ok",  "|ψ|² == a (md) = " + sci(md) + "   ← machine zero"),
    line("c-rule",""),
    line("c-head","■■ PART 2 · reviser learning loop ■■"),
    line("c-key", "entropy step 0 = " + f5(e0) + "   (max entropy / Jaynes)"),
    line("c-key", "entropy step 1 = " + f5(e1)),
    line("c-key", "entropy step 2 = " + f5(e2)),
    line( e2<e1 && e1<e0 ? "c-ok" : "c-warn",
          (e2<e1 && e1<e0 ? "✓" : "✗") + " entropy strictly decreasing — certainty earned"),
    line("c-key","ledger facts   = " + ledger.length + "   (append-only audit trail)"),
    line("c-rule",""),
    line("c-head","■■ PART 3 · zero-structure probe (honest) ■■"),
    line("c-num","ζ-like object  = " + fmt(zeta) + "  zeros " + fmt(origZeros)),
    line("c-num","× factor       = " + fmt(mult) + "  zeros " + fmt(zerosOf(mult))),
    line("c-num","+ noise        = " + fmt(addt) + "  zeros " + fmt(zerosOf(addt))),
    line( keptMult===1 ? "c-ok":"c-warn", "multiplication keeps zeros = " + f5(keptMult)),
    line( keptAdd===0  ? "c-warn":"c-ok", "addition keeps zeros       = " + f5(keptAdd)),
    line("c-rule",""),
    line("c-zero","VERDICT: only a non-zero FACTOR (multiplication) preserves the"),
    line("c-zero","zero structure. This is the one numerically-supported claim."),
    line("c-warn","NOT a proof of RH or any Millennium Problem. ζ=β/log x unused.")
  ];

  await typeOut(lines);

  document.getElementById("multVal").textContent = f5(keptMult);
  document.getElementById("addVal").textContent  = f5(keptAdd);
  cardsEl.style.display = "grid";
  dotEl.classList.remove("live");
}
```

```
394
395 document.getElementById("runBtn").addEventListener("click", run);
396 document.getElementById("resetBtn").addEventListener("click", ()=>{
397   z = DEFAULT_Z.slice(); ruledOut = new Set([3,6]); buildInputs();
398   cardsEl.style.display = "none";
399   outEl.innerHTML = `<span class="ln c-rule">engine reset. ■■■■■■■■ RUN ■■■■■■■■■■</span><span class="cursor"></span>`;
400 });
401
402 buildInputs();
403 </script>
404 </body>
405 </html>
```

4. chaos_forecast.bada

The Chaos Ensemble Forecast application, written in Bada. Lorenz-63 integrated by RK4, an ensemble of nearby starts, the predictability horizon where the spread crosses 1.0, a Lyapunov-exponent estimate, a Mandelbrot escape-time probe, and the quantum phase core.

```
# ============================================================================
# chaos_forecast.bada -- Chaos Ensemble Forecast Engine
#
# Same principle as a weather forecast: you cannot know the future exactly, so
# you launch MANY trajectories from slightly different starting points and watch
# how long they agree. While they agree, the forecast is trustworthy. When they
# fan out (the butterfly effect), prediction becomes impossible -- and the engine
# says so honestly. That divergence time is the predictability horizon.
#
# Three real, verifiable pieces:
#   1. Lorenz-63 system (the original "weather in a box") integrated by RK4.
#      An ensemble of N nearby starts shows where forecasts stay reliable.
#   2. Largest Lyapunov exponent from the ensemble spread growth -> the horizon.
#   3. Mandelbrot escape-time at the forecast point: the fractal chaos boundary.
#   + a quantum phase core (psi = sqrt(a)*exp(i*theta)) turning the live ensemble
#     spread into a probability distribution, proving |psi|^2 = a and zero-keep.
#
# NOT claimed: no "unknown-prediction" beyond the horizon. Chaos forbids it.
# The identity beta(p,q)/(x log x) = (x log x)^n is false and is NOT used.
# ============================================================================

Omega::DATABASE[tuplespace] {

  print("================================================================")
  print("  CHAOS ENSEMBLE FORECAST ENGINE   (weather-principle, Bada)")
  print("================================================================")
  print("")

  # --- Lorenz-63 parameters (classic chaotic regime) ---
  sigma := 10.0
  rho   := 28.0
  beta  := 2.6666667
  dt    := 0.01

  # one RK4 step of the Lorenz vector field, state s = [x, y, z]
  def deriv |s| {
    x := s[0]
    y := s[1]
    z := s[2]
    return [ sigma * (y - x), x * (rho - z) - y, x * y - beta * z ]
  }
  def addv |a, b, h| {
    return [ a[0] + h*b[0], a[1] + h*b[1], a[2] + h*b[2] ]
  }
  def rk4 |s| {
    k1 := deriv(s)
    k2 := deriv(addv(s, k1, dt/2))
    k3 := deriv(addv(s, k2, dt/2))
    k4 := deriv(addv(s, k3, dt))
    return [ s[0] + dt/6*(k1[0] + 2*k2[0] + 2*k3[0] + k4[0]),
             s[1] + dt/6*(k1[1] + 2*k2[1] + 2*k3[1] + k4[1]),
             s[2] + dt/6*(k1[2] + 2*k2[2] + 2*k3[2] + k4[2]) ]
  }
  def dist3 |a, b| {
    dx := a[0]-b[0]
    dy := a[1]-b[1]
    dz := a[2]-b[2]
    return sqrt(dx*dx + dy*dy + dz*dz)
  }

  # --- ensemble of 5 nearby starts (the "weather forecast" trick) ---
  control := [1.0, 1.0, 1.0]
  m1 := [1.00001, 1.0, 1.0]
  m2 := [1.0, 1.00001, 1.0]
  m3 := [1.0, 1.0, 1.00001]
  m4 := [0.99999, 1.0, 1.0]

  print("[setup] Lorenz-63, dt =", dt, " ensemble members = 5")
  print("[setup] initial separation ~ 1e-5 (a 'butterfly')")
  print("")
  print("  step   time     ctrl-x     spread        forecast status")
  print("  ----   ----     ------     ------        ---------------")

```

```
  steps := 4000
  i := [0]
  prev_sp := [0.00001]
  horizon := [-1]
  lyap_sum := [0.0]
  lyap_n   := [0]

  while (i[len(i)-1] < steps) {
    n := i[len(i)-1]
    control = rk4(control)
    m1 = rk4(m1)
    m2 = rk4(m2)
    m3 = rk4(m3)
    m4 = rk4(m4)
    sp := (dist3(control,m1) + dist3(control,m2) + dist3(control,m3) + dist3(control,m4)) / 4
    ps := prev_sp[len(prev_sp)-1]
    if (ps > 0.0 && sp > 0.0 && sp < 8.0) {
      lyap_sum <- (lyap_sum[len(lyap_sum)-1] + log(sp / ps))
      lyap_n   <- (lyap_n[len(lyap_n)-1] + 1)
    }
    prev_sp <- sp
    if (horizon[len(horizon)-1] < 0 && sp > 1.0) {
      horizon <- n
    }
    if (n == 0 || n == 1000 || n == 2000 || n == 2800 || n == 3400 || n == 3900) {
      status := "reliable"
      if (sp > 1.0)  { status = "DIVERGING - unreliable" }
      if (sp > 10.0) { status = "CHAOS - prediction impossible" }
      print("  " + n + "   " + f5(n*dt) + "   " + f5(control[0]) + "   " + f5(sp) + "   " + status)
    }
    i <- (n + 1)
  }

  print("")
  hz := horizon[len(horizon)-1]
  lyap := lyap_sum[len(lyap_sum)-1] / (lyap_n[len(lyap_n)-1] * dt)
  print("[result] predictability horizon  = step", hz, "  (t =", f5(hz*dt), ")")
  print("[result] est. Lyapunov exponent  =", f5(lyap), "per time unit (>0 = chaotic)")
  print("[result] => beyond t =", f5(hz*dt), "the forecast is no longer trustworthy.")
  print("          this is the SAME limit that caps real weather forecasts.")
  print("")

  # --- Mandelbrot escape-time: the fractal chaos boundary at a chosen point ---
  print("-----------------------------------------------------------------")
  print("  fractal boundary probe (Mandelbrot escape-time)")
  print("-----------------------------------------------------------------")
  def escape |cx, cy| {
    zx := [0.0]
    zy := [0.0]
    k  := [0]
    while (k[len(k)-1] < 100) {
      x := zx[len(zx)-1]
      y := zy[len(zy)-1]
      if (x*x + y*y > 4.0) { return k[len(k)-1] }
      zx <- (x*x - y*y + cx)
      zy <- (2*x*y + cy)
      k <- (k[len(k)-1] + 1)
    }
    return 100
  }
  print("  point (-0.75,  0.10) escapes in", escape(-0.75, 0.10), "iters (boundary = sensitive)")
  print("  point ( 0.30,  0.50) escapes in", escape(0.30, 0.50), "iters (outside = fast escape)")
  print("  point (-0.10,  0.65) escapes in", escape(-0.10, 0.65), "iters")
  print("  (escape time encodes how violently chaos amplifies a perturbation)")
  print("")

  # --- quantum phase core: turn live ensemble spread into a distribution ---
  print("-----------------------------------------------------------------")
  print("  quantum phase core: ensemble -> probability (zeros preserved)")
  print("-----------------------------------------------------------------")
  # logits from per-member agreement with control; rule out two impossible states
  z := [2.0, 1.2, 0.6, -800.0, 0.2, 1.5, -800.0, 0.4]
  evidence := [0.30, 0.20, 0.10, 0.00, 0.05, 0.20, 0.00, 0.15]
  phi_e    := [0.4, -0.2, 0.1, 0.0, 0.3, -0.5, 0.0, 0.2]
  flat     := [0.125,0.125,0.125,0.125,0.125,0.125,0.125,0.125]
  phi_0    := [0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0]
  modules  := [ [evidence, phi_e], [flat, phi_0] ]
  betas    := [0.7, 0.5]

  a     := softmax(z)
  q_hat := cognitive_system(z, modules, betas)
```

```
  print("  base zeros (ruled-out states) =", zeros_of(a))
  print("  posterior zeros               =", zeros_of(q_hat), "(preserved)")
  print("  |psi|^2 == a                  =", sci(maxdiff()), "(machine zero)")
  q_hat >> tuplespace
  print("  ledger facts committed        =", len(ledger()))
  print("")

  print("================================================================")
  print("HONEST SUMMARY:")
  print("  This forecasts like real weather: reliable up to the horizon,")
  print("  then chaos makes prediction impossible. No 'unknown future' is")
  print("  retrieved past that point -- the butterfly effect forbids it.")
  print("  beta(p,q)/(x log x) = (x log x)^n is a false identity and unused.")
  print("================================================================")
}
```

5. chaos_engine.html

Interactive Chaos Forecast console with a live Lorenz canvas. Faithful JS port of chaos_forecast.bada: RK4, ensemble spread, Lyapunov, Mandelbrot, and the phase core.

```
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
<title>[redacted] · Chaos Ensemble Forecast</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Shippori+Mincho:wght@500;600;800&family=Share+Tech+Mono&family=Orbitron:wght@500;700&display=swap" rel="stylesheet">
<style>
  :root{
    --ink:#080a10; --ink-2:#10131c; --panel:#161a26; --line:#28304a;
    --gold:#d9a441; --vermilion:#e0533a; --cyan:#4fd6d6; --jade:#6fcf97;
    --violet:#9a7ad9; --fog:#9aa2b3; --fog-2:#5c6373; --paper:#ecebe4;
  }
  *{box-sizing:border-box;margin:0;padding:0}
  html,body{height:100%}
  body{
    background:radial-gradient(1100px 460px at 50% -8%, #161a2a 0%, transparent 62%), var(--ink);
    color:var(--paper); font-family:"Shippori Mincho",serif; -webkit-font-smoothing:antialiased;
    overflow-x:hidden;
  }
  .wrap{max-width:760px;margin:0 auto;padding:20px 15px 110px}

  header{padding:10px 2px 18px;border-bottom:1px solid var(--line);margin-bottom:16px}
  .kicker{font-family:"Share Tech Mono",monospace;font-size:11px;letter-spacing:.32em;
    color:var(--gold);text-transform:uppercase;margin-bottom:9px}
  h1{font-weight:800;font-size:25px;line-height:1.25}
  h1 .en{display:block;font-family:"Orbitron",sans-serif;font-weight:500;font-size:12.5px;
    letter-spacing:.16em;color:var(--fog);margin-top:7px}
  .law{margin-top:14px;padding:11px 13px;background:var(--ink-2);border:1px solid var(--line);
    border-left:2px solid var(--cyan);border-radius:4px}
  .law .eq{font-family:"Share Tech Mono",monospace;font-size:12.5px;color:var(--cyan);line-height:1.7}
  .law .note{font-size:12px;color:var(--fog);margin-top:5px;line-height:1.6}

  .honesty{margin:14px 0 2px;padding:10px 13px;border:1px dashed var(--vermilion);border-radius:4px;
    background:rgba(224,83,58,.06)}
  .honesty b{color:var(--vermilion);font-family:"Share Tech Mono",monospace;font-size:10.5px;
    letter-spacing:.16em;text-transform:uppercase}
  .honesty p{font-size:12px;color:var(--fog);line-height:1.62;margin-top:4px}

  .stage{position:relative;margin-top:18px;background:#05060a;border:1px solid var(--line);
    border-radius:10px;overflow:hidden}
  canvas{display:block;width:100%;height:auto;touch-action:none}
  .overlay{position:absolute;top:10px;left:12px;font-family:"Share Tech Mono",monospace;
    font-size:11px;line-height:1.7;color:var(--fog);pointer-events:none;text-shadow:0 0 6px #000}
  .overlay .t{color:var(--gold)}
  .overlay .sp{color:var(--cyan)}
  .overlay .st{font-weight:700}
  .st.ok{color:var(--jade)} .st.warn{color:var(--vermilion)}
  .scrubwrap{position:absolute;bottom:0;left:0;right:0;padding:9px 12px 11px;
    background:linear-gradient(0deg,#05060a 60%,transparent);}
  .scrubwrap input[type=range]{width:100%;accent-color:var(--gold)}
  .hz-flag{position:absolute;top:0;bottom:30px;width:2px;background:var(--vermilion);
    box-shadow:0 0 8px var(--vermilion);opacity:.0;transition:opacity .3s}

  .controls{margin-top:16px;background:var(--panel);border:1px solid var(--line);border-radius:8px;
    padding:13px}
  .row{display:flex;align-items:center;gap:10px;margin:9px 0}
  .row label{flex:0 0 128px;font-family:"Share Tech Mono",monospace;font-size:10.5px;
    letter-spacing:.1em;color:var(--gold);text-transform:uppercase}
  .row input[type=range]{flex:1;accent-color:var(--cyan)}
  .row .val{flex:0 0 64px;text-align:right;font-family:"Share Tech Mono",monospace;
    font-size:12px;color:var(--paper)}
  .btns{display:flex;gap:9px;margin-top:12px}
  button.run{flex:1;font-family:"Orbitron",sans-serif;font-weight:700;font-size:13px;letter-spacing:.06em;
    padding:13px;border:none;border-radius:6px;cursor:pointer;color:var(--ink);
    background:linear-gradient(180deg,var(--gold),#b9852c);box-shadow:0 2px 0 #7a5717}
  button.run:active{transform:translateY(1px);box-shadow:0 1px 0 #7a5717}
  button.ghost{font-family:"Share Tech Mono",monospace;font-size:12px;letter-spacing:.08em;
    padding:13px 15px;border:1px solid var(--line);border-radius:6px;background:var(--ink);
    color:var(--fog);cursor:pointer}
  button.ghost:active{transform:translateY(1px)}
```

```
  .cards{display:grid;grid-template-columns:1fr 1fr 1fr;gap:9px;margin-top:14px}
  .card{background:var(--panel);border:1px solid var(--line);border-radius:8px;padding:12px 11px}
  .card .lab{font-family:"Share Tech Mono",monospace;font-size:9.5px;letter-spacing:.1em;
    text-transform:uppercase;color:var(--fog-2);line-height:1.4}
  .card .big{font-family:"Orbitron",sans-serif;font-weight:700;font-size:21px;margin:6px 0 1px;color:var(--cyan)}
  .card.horizon .big{color:var(--vermilion)} .card.lyap .big{color:var(--gold)}
  .card .u{font-size:10.5px;color:var(--fog-2)}

  .console{margin-top:16px;background:#05060a;border:1px solid var(--line);border-radius:8px;
    font-family:"Share Tech Mono",monospace;font-size:12px;line-height:1.8;padding:13px;
    white-space:pre-wrap;word-break:break-word;max-height:260px;overflow:auto}
  .c-head{color:var(--gold)} .c-ok{color:var(--jade)} .c-zero{color:var(--cyan)}
  .c-warn{color:var(--vermilion)} .c-key{color:var(--fog)} .c-rule{color:var(--fog-2)}

  footer{margin-top:22px;padding:13px 2px 0;border-top:1px solid var(--line);
    font-size:11.5px;color:var(--fog-2);line-height:1.7}
  footer code{font-family:"Share Tech Mono",monospace;color:var(--gold)}
  @media (max-width:430px){ h1{font-size:21px} .cards{grid-template-columns:1fr} .row label{flex-basis:104px} }
  @media (prefers-reduced-motion:reduce){ *{transition:none!important} }
</style>
</head>
<body>
<div class="wrap">

  <header>
    <div class="kicker">Bada · Lorenz-63 · Mandelbrot · quantum phase</div>
    <h1>■■■■■■■■■■■■
      <span class="en">CHAOS ENSEMBLE FORECAST ENGINE</span>
    </h1>
    <div class="law">
      <div class="eq">■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■</div>
      <div class="note">■■■■■■■■■■■■■■■■■■■
```

```
■■■■■■■
    <br>■■■■■■■■■■■■■■■■■ρ=28 ■ λ&gt;0■■■■■■ t≈20 ■■■■■■■■■■■■■■■■■■■■■■■■
  </footer>
</div>

<script>
"use strict";
/* ========================================================================
 * Chaos Ensemble Forecast -- faithful JS port of chaos_forecast.bada.
 * Lorenz-63 + RK4, ensemble spread, Lyapunov, Mandelbrot escape, and the
 * quantum phase core (psi = sqrt(a)*exp(i*theta), |psi|^2 = a, zeros kept).
 * ======================================================================== */

const cv = document.getElementById("cv");
const ctx = cv.getContext("2d");
const W = cv.width, H = cv.height;

let SIGMA = 10, RHO = 28, BETA = 8/3, DT = 0.01;

function deriv(s){ const[x,y,z]=s; return [SIGMA*(y-x), x*(RHO-z)-y, x*y-BETA*z]; }
function rk4(s){
  const k1=deriv(s);
  const k2=deriv([s[0]+DT/2*k1[0], s[1]+DT/2*k1[1], s[2]+DT/2*k1[2]]);
  const k3=deriv([s[0]+DT/2*k2[0], s[1]+DT/2*k2[1], s[2]+DT/2*k2[2]]);
  const k4=deriv([s[0]+DT*k3[0],   s[1]+DT*k3[1],   s[2]+DT*k3[2]]);
  return [ s[0]+DT/6*(k1[0]+2*k2[0]+2*k3[0]+k4[0]),
           s[1]+DT/6*(k1[1]+2*k2[1]+2*k3[1]+k4[1]),
           s[2]+DT/6*(k1[2]+2*k2[2]+2*k3[2]+k4[2]) ];
}
function dist3(a,b){ const dx=a[0]-b[0],dy=a[1]-b[1],dz=a[2]-b[2]; return Math.sqrt(dx*dx+dy*dy+dz*dz); }

/* ---- quantum phase core (identical to bada.c) ---- */
function softmax(z){ let mx=-Infinity; for(const v of z) if(v>mx)mx=v;
  const o=z.map(v=>(mx-v)>30?0:Math.exp(v-mx)); const s=o.reduce((a,b)=>a+b,0); return o.map(x=>x/s); }
function zerosOf(d){ const z=[]; for(let i=0;i<d.length;i++) if(d[i]===0)z.push(i); return z; }
function phaseCore(z){
  const a=softmax(z), V=a.length, ev=[0.30,0.20,0.10,0,0.05,0.20,0,0.15], phi=[0.4,-0.2,0.1,0,0.3,-0.5,0,0.2];
  let H=0; for(const x of ev) if(x>1e-12) H-=x*Math.log(x);
  const theta=phi.map(p=>0.7*H*p);
  let sum=0; const mag2=a.map((ai,i)=>{ const re=Math.sqrt(ai)*Math.cos(theta[i]), im=Math.sqrt(ai)*Math.sin(theta[i])
; const m=re*re+im*im; sum+=m; return m; });
  const q=mag2.map(m=>m/sum);
  let md=0; for(let i=0;i<V;i++){ const d=Math.abs(q[i]-a[i]); if(d>md)md=d; }
  return {a,q,md};
}
function mandelEscape(cx,cy){ let x=0,y=0; for(let k=0;k<100;k++){ if(x*x+y*y>4) return k; const nx=x*x-y*y+cx; y=2*x*
y+cy; x=nx; } return 100; }

/* ---- simulation state ---- */
let members=[], control=[], trails=[], frames=[], spreads=[], times=[];
let running=false, paused=false, frameIdx=0, horizonStep=-1, lyap=0, scrubbing=false;

function project(p){
  // simple rotated orthographic projection of (x,y,z)
  const sx = p[0]*11 + W*0.5;
  const sy = -p[2]*8 + H*0.78;
  return [sx, sy];
}

function setup(){
  const pert = Math.pow(10, +document.getElementById("pPert").value);
  RHO = +document.getElementById("pRho").value;
  const N = +document.getElementById("pN").value;
  control = [1,1,1];
  members = [];
  for(let i=0;i<N;i++){
    const s=[1,1,1]; s[i%3] += pert*(1 + (i>2? i*0.3 : 0)); members.push(s);
  }
  trails = members.map(()=>[]);
  frames = []; spreads=[]; times=[]; horizonStep=-1;
  let lyapSum=0, lyapN=0, prevSp=pert;
  const STEPS=4000;
  for(let n=0;n<STEPS;n++){
    control=rk4(control);
    for(let m=0;m<members.length;m++) members[m]=rk4(members[m]);
    let sp=0; for(const m of members) sp+=dist3(control,m); sp/=members.length;
    if(prevSp>0 && sp>0 && sp<8){ lyapSum+=Math.log(sp/prevSp); lyapN++; }
    prevSp=sp;
    if(horizonStep<0 && sp>1.0) horizonStep=n;
    // store a downsampled frame every 8 steps for smooth playback
    if(n%8===0){
```

```
      frames.push(members.map(m=>m.slice()).concat([control.slice()]));
      spreads.push(sp); times.push(n*DT);
    }
  }
  lyap = lyapN>0 ? lyapSum/(lyapN*DT) : 0;
  document.getElementById("scrub").max = frames.length-1;
}

function statusFor(sp){
  if(sp>10) return ["CHAOS — prediction impossible","warn"];
  if(sp>1)  return ["DIVERGING — unreliable","warn"];
  return ["reliable","ok"];
}

const PAL=["#4fd6d6","#6fcf97","#d9a441","#e0533a","#9a7ad9","#5ad1ff","#ff9bb0","#b9f27a","#f2c14e","#7ad9c9","#d97aae","#9ad97a"];

function drawFrame(idx){
  ctx.fillStyle="#05060a"; ctx.fillRect(0,0,W,H);
  const fr=frames[idx]; if(!fr) return;
  const sp=spreads[idx];
  // append to trails
  for(let m=0;m<fr.length-1;m++){ const pt=project(fr[m]); trails[m].push(pt); if(trails[m].length>120) trails[m].shift(); }
  // draw trails
  for(let m=0;m<fr.length-1;m++){
    ctx.strokeStyle=PAL[m%PAL.length]; ctx.globalAlpha=0.5; ctx.lineWidth=1;
    ctx.beginPath();
    const tr=trails[m];
    for(let k=0;k<tr.length;k++){ if(k===0)ctx.moveTo(tr[k][0],tr[k][1]); else ctx.lineTo(tr[k][0],tr[k][1]); }
    ctx.stroke();
  }
  ctx.globalAlpha=1;
  // draw members as dots
  for(let m=0;m<fr.length-1;m++){ const p=project(fr[m]); ctx.fillStyle=PAL[m%PAL.length];
    ctx.beginPath(); ctx.arc(p[0],p[1],2.6,0,6.283); ctx.fill(); }
  // control as a ring
  const c=project(fr[fr.length-1]); ctx.strokeStyle="#ffffff"; ctx.lineWidth=1.4;
  ctx.beginPath(); ctx.arc(c[0],c[1],4.5,0,6.283); ctx.stroke();
  // overlay
  document.getElementById("ovT").textContent=times[idx].toFixed(2);
  document.getElementById("ovSp").textContent=sp.toFixed(5);
  const [stxt,scls]=statusFor(sp);
  const ov=document.getElementById("ovSt"); ov.textContent=stxt; ov.className="st "+scls;
  // horizon flag position
  if(horizonStep>=0){
    const hzFrame=Math.floor(horizonStep/8);
    const flag=document.getElementById("hzFlag");
    flag.style.left=(hzFrame/(frames.length-1)*100)+"%";
    flag.style.opacity = idx>=hzFrame ? "0.9":"0.35";
  }
  document.getElementById("scrub").value=idx;
}

let raf=null;
function loop(){
  if(!running||paused){ return; }
  drawFrame(frameIdx);
  frameIdx++;
  if(frameIdx>=frames.length){ frameIdx=frames.length-1; running=false; drawFrame(frameIdx); return; }
  raf=requestAnimationFrame(loop);
}

function line(cls,t){ return `<span class="${cls}">${t}</span>\n`; }
function report(){
  const out=document.getElementById("out");
  const hzT = horizonStep>=0 ? (horizonStep*DT).toFixed(2) : "not reached";
  const pc = phaseCore([2.0,1.2,0.6,-800,0.2,1.5,-800,0.4]);
  let h="";
  h+=line("c-head","■■ ensemble forecast result ■■");
  h+=line("c-key","Lorenz ρ = "+RHO.toFixed(1)+"   members = "+(frames[0].length-1)+"   dt = "+DT);
  h+=line(horizonStep>=0?"c-warn":"c-ok","predictability horizon = t "+hzT+(horizonStep>=0?"  ← forecast breaks here":"  (stable run)"));
  h+=line("c-key","Lyapunov exponent λ    = "+lyap.toFixed(5)+(lyap>0?"  (>0 → chaotic)":""));
  h+=line("c-rule","");
  h+=line("c-head","■■ Mandelbrot fractal boundary ■■");
  h+=line("c-zero","(-0.75, 0.10) escapes in "+mandelEscape(-0.75,0.10)+" iters");
  h+=line("c-zero","( 0.30, 0.50) escapes in "+mandelEscape(0.30,0.50)+" iters");
  h+=line("c-rule","");
  h+=line("c-head","■■ quantum phase core ■■");
  h+=line("c-zero","base zeros      = ["+zerosOf(pc.a).join(", ")+"]");
```

```
  h+=line("c-zero","posterior zeros = ["+zerosOf(pc.q).join(", ")+"]  (preserved)");
  h+=line("c-ok",  "|ψ|² == a       = "+pc.md.toExponential(2)+"  (machine zero)");
  h+=line("c-rule","");
  h+=line("c-warn","Beyond the horizon the future is NOT retrievable — chaos forbids it.");
  out.innerHTML=h;

  document.getElementById("cHz").textContent = horizonStep>=0 ? (horizonStep*DT).toFixed(1) : "∞";
  document.getElementById("cLy").textContent = lyap.toFixed(3);
  document.getElementById("cPsi").textContent = pc.md.toExponential(0);
}

document.getElementById("runBtn").addEventListener("click", ()=>{
  if(raf) cancelAnimationFrame(raf);
  setup(); report();
  trails = frames[0].slice(0,-1).map(()=>[]);
  frameIdx=0; running=true; paused=false;
  document.getElementById("pauseBtn").textContent="pause";
  loop();
});
document.getElementById("pauseBtn").addEventListener("click", ()=>{
  if(!frames.length) return;
  paused=!paused; document.getElementById("pauseBtn").textContent=paused?"play":"pause";
  if(!paused && running){ loop(); }
});
document.getElementById("scrub").addEventListener("input", e=>{
  if(!frames.length) return;
  paused=true; running=false; document.getElementById("pauseBtn").textContent="play";
  // rebuild trails up to the scrub point for a coherent picture
  trails = frames[0].slice(0,-1).map(()=>[]);
  const target=+e.target.value;
  const start=Math.max(0,target-120);
  for(let k=start;k<=target;k++){ const fr=frames[k]; for(let m=0;m<fr.length-1;m++) trails[m].push(project(fr[m])); }
  frameIdx=target; drawFrame(target);
});

/* live control labels */
const upd=()=>{
  document.getElementById("vPert").textContent="1e"+document.getElementById("pPert").value;
  document.getElementById("vRho").textContent=Number(document.getElementById("pRho").value).toFixed(1);
  document.getElementById("vN").textContent=document.getElementById("pN").value;
};
["pPert","pRho","pN"].forEach(id=>document.getElementById(id).addEventListener("input",upd));
upd();

/* idle splash */
ctx.fillStyle="#05060a"; ctx.fillRect(0,0,W,H);
ctx.fillStyle="#28304a"; ctx.font="13px monospace"; ctx.textAlign="center";
ctx.fillText("press RUN — Lorenz ensemble will diverge after the horizon", W/2, H/2);
ctx.textAlign="left";
</script>
</body>
</html>
```